Academic Year 2025 – 2026
University: University of Pisa / Sant'Anna School of Advanced Studies
Course: Statistical Learning & Large Data
Supervisor(s): Prof.ssa Francesca Chiaromonte

SLLD - Module 2

Regularized and Stable Lexical Classification in High-Dimensional Semantic Feature Spaces

Michela Deodati & Leonardo Pratelli

Abstract

This project reformulates the semantic rating dataset of Binder et al. (Binder et al., 2016) as a supervised multi-class learning problem. English lexical items, described by 65 experiential semantic attributes, are classified into three lexical-semantic classes: *Thing*, *Action*, and *Property*. After data cleaning, imputation, standardization, and exploratory analysis, the original feature space is used to fit baseline multinomial models and regularized alternatives, with attention to class imbalance, multicollinearity, and interpretability. The analysis is then extended to a high-dimensional representation obtained through quadratic terms and pairwise interactions. Feature screening, LASSO, and Elastic Net are used to control dimensionality, reduce overfitting, and identify relevant semantic predictors. Model performance is assessed through accuracy, balanced accuracy, macro-F1, class-wise recall, and confusion matrices. The project evaluates whether high-dimensional statistical learning can improve lexical-class prediction while preserving a meaningful semantic interpretation of the selected features.

Keywords: Statistical learning, high-dimensional data, semantic features, regularization, feature screening, lexical classification

This report was prepared for the Statistical Learning for Large Data exam. The work has roots on the previous work "Unveiling Word Meaning: A statistical study on primitive semantic features", developed by Davide Testa, *and reformulates it as a high-dimensional statistical learning study focused on large feature spaces, regularization, feature screening, class imbalance and stability.*

Contents

1 Introduction **4**

2 Dataset **4**

2.1 I. Data Preprocessing 6

2.2 II. Unsupervised exploration of the semantic space 8

2.3 III. Supervised classification on the original feature space 13

2.4 IV. Alternative classifiers 17

2.5 V. High-dimensional expansion and feature screening 21

2.6 VI. Final model comparison and interpretation 26

3 Results **27**

4 Discussion **29**

5 Conclusion **32**

References **34**

A Original clustering and biclustering analyses **36**

B Detailed explanation of the Python preprocessing pipeline **37**

B.1 Phase 1.1: Initial dataset cleaning 38

B.2 Phase 1.2: Stratified train–test split 42

B.3 Phase 1.3: Imputation and standardization 46

B.4 Overall logic of the preprocessing pipeline 51

C Detailed explanation of the Python analysis pipeline **52**

C.1 Phase 2: Unsupervised exploration of the semantic space 52

C.2 Phase 3.1: Supervised data loading and preliminary checks 55

C.3 Phase 3.2: Multinomial logistic baseline 57

C.4 Phase 3.3: Multicollinearity diagnostics 59

C.5 Phase 3.4: Feature screening 61

C.6 Phase 3.5: Regularized multinomial models 62

C.7 Phase 3.6: Sparsity analysis 63

C.8 Phase 3.7: Final model evaluation 64

C.9 Phase 3-extra: Alternative classifiers 65
C.10 Overall logic of the post-preprocessing pipeline 66

D Additional tables and figures **68**

D.1 Preprocessing and supervised data checks 68
D.2 PCA and unsupervised structure 68
D.3 Baseline supervised model 71
D.4 Multicollinearity diagnostics 72
D.5 Feature screening 74
D.6 Regularized models and sparsity analysis 76
D.7 Final model evaluation and alternative classifiers 78
D.8 Cluster-label supervised analysis 80

1 Introduction

The present work builds on the previous project ***Unveiling Word Meaning: A statistical study on primitive semantic features***, developed by Davide Testa(Testa, 2026).. Testa's work was concerned with the statistical analysis of semantic representations derived from the dataset introduced by Binder et al.(Binder et al., 2016). In the original project, Testa approached the dataset from the perspective of lexical semantics, psycholinguistics, and neurocognitive theories of meaning representation. His main objective was to assess whether a componential, brain-inspired representation of word meaning could capture meaningful semantic structures and support the distinction between broad lexical classes. Testa's original analysis combined unsupervised and supervised methods. The unsupervised part used correlation analysis, k-means clustering, hierarchical clustering, biclustering(Csardi, 2023), and PCA to examine whether semantic similarity structures could emerge from the feature space alone. These analyses suggested that the semantic attributes were not randomly distributed, but organized into interpretable domains, with PCA highlighting a relevant opposition between more concrete and more abstract semantic dimensions. The supervised part addressed whether lexical classes could be predicted from semantic information alone, using multinomial regression with LASSO regularization and feature selection. Overall, Testa's work showed that primitive semantic features carry enough information both to recover meaningful semantic groupings and to support the distinction between broad lexical classes. Our project takes this original analysis as a methodological and conceptual starting point, but shifts the focus toward supervised high-dimensional statistical learning, regularization, feature screening, and predictive evaluation.

2 Dataset

The starting point of this project is the dataset used in Testa's original work, his study was based on the semantic ratings introduced by Binder et al. (Binder et al., 2016) and used them to investigate whether a brain-based componential representation of meaning could reveal interpretable semantic structures and support the distinction between broad lexical classes. In that framework, lexical items are represented through primitive experiential attributes: each feature measures the relevance of a specific semantic component to the meaning of a word. These components span several domains of experience, including sensory, motor, spatial, temporal, causal, social, affective, drive-related, attentional and cognitive dimensions. In the final version used by Testa, the dataset was reduced to the variables directly relevant to the semantic analysis. After removing duplicated units and excluding auxiliary variables not needed for the componential study, the data consisted of 534 English lexical items and 66 variables: 65 quantitative semantic features plus one qualitative variable, `Type`. The variable `Type` assigns each lexical item to one of three lexical-semantic classes: *Thing*, *Action*, and *Property*. These classes correspond, respectively, to nouns, verbs, and adjectives or property-like items. The class distribution is strongly unbalanced, as shown in Table 1.

Class	Number of items	Percentage
Thing	434	81.27%
Action	56	10.49%
Property	44	8.24%
Total	**534**	**100.00%**

Table 1 – Class distribution of the lexical-semantic target variable.

This distribution is an important numerical property of the dataset. The majority class, *Thing*, represents more than four fifths of the observations. The imbalance ratio between the largest and smallest class is

$$\frac{434}{44} \approx 9.86,$$

while the ratio between *Thing* and *Action* is

$$\frac{434}{56} \approx 7.75.$$

As a consequence, a trivial classifier always predicting *Thing* would already obtain a raw accuracy of

$$\frac{434}{534} \approx 81.27\%.$$

However, the same classifier would have zero recall for both *Action* and *Property*. For this reason, the dataset cannot be evaluated using accuracy alone. The class imbalance directly motivates the use of balanced accuracy, macro-F1, class-wise recall and confusion matrices in the supervised evaluation.In its original form, the dataset is moderate-dimensional. The number of predictors is

$$p_{\text{orig}} = 65,$$

with

$$\frac{p_{\text{orig}}}{n} = \frac{65}{534} \approx 0.12, \qquad \frac{n}{p_{\text{orig}}} = \frac{534}{65} \approx 8.22.$$

Thus, before feature expansion, the number of observations is substantially larger than the number of semantic predictors. This original matrix is used to fit baseline models and regularized models on the native semantic representation. Our project takes the same semantic matrix as a starting point but reformulates it as a supervised statistical learning problem. The original 534×65 representation is first used to build baseline and regularized models on the native semantic feature space. The analysis is then extended by transforming the original predictors into a high-dimensional feature space containing the original variables, their quadratic terms and all pairwise interactions. Formally, if

$$X = (X_1, X_2, \ldots, X_{65})$$

denotes the original vector of semantic attributes, the expanded representation includes

$$X_1, \ldots, X_{65},$$

$$X_1^2, \ldots, X_{65}^2,$$

and all interaction terms

$$X_j X_k, \qquad j < k.$$

The number of pairwise interactions is

$$\binom{65}{2} = 2080,$$

so the transformed feature space contains

$$65 + 65 + 2080 = 2210$$

predictors. The dataset is therefore transformed from an original matrix

$$X_{\text{orig}} \in \mathbb{R}^{534\times 65}$$

into a high-dimensional matrix

$$X_{\text{HD}} \in \mathbb{R}^{534\times 2210}.$$

This transformation changes the statistical nature of the problem. In the original feature space, the number of observations is larger than the number of predictors. After expansion, instead, the number of predictors exceeds the number of lexical items:

$$p = 2210 > n = 534.$$

The project therefore moves from a moderate-dimensional semantic classification problem to a high-dimensional learning setting. This makes ordinary unregularized multinomial regression unsuitable and motivates the use of regularization, dimensionality reduction and feature screening. At the same time, the expanded representation remains semantically interpretable: quadratic terms capture nonlinear effects of individual semantic dimensions, while interaction terms capture combined semantic profiles, such as the joint contribution of motion and body-related features for actions or social and cognitive features for abstract meanings.

2.1 I. Data Preprocessing

The preprocessing phase is designed to produce a clean, reproducible, and statistically usable version of the rating dataset before any exploratory or predictive analysis is performed. The guiding principle of this phase is to preserve as much valid lexical information as possible while avoiding data leakage between training and test data. We start from the 65-feature version of the dataset, in which each observation corresponds to a lexical entry and each predictor corresponds to an experiential semantic attribute. The dataset contains three structural columns, namely `entry_id`, `word`, and `target_word_class`, plus 65 semantic features. The target variable is represented by three lexical-semantic classes: *noun*, *verb*, and *adjective*. In the first preprocessing script, we verify that all required columns are present, that the expected number of semantic predictors is equal to 65, and that the target variable contains only valid class labels. During the initial cleaning step, we do not apply any statistical transformation to the predictors. We only perform structural checks. Exact duplicated rows are searched for and removed, none was found. We also inspect duplicated word forms. This second check identifies two rows corresponding to the word *used*, which appears once as an adjective and once as a verb. These rows are not removed, because the statistical unit of the analysis is the lexical entry rather than the orthographic word form alone. A repeated word form with a different lexical-semantic class is therefore considered a valid observation. We then examine missing values feature by feature. As shown in Table 2, missingness is limited to four semantic predictors: `complexity`, `practice`, `caused`, and `drive`. The feature `complexity` contains the largest number of missing values, with 101 missing entries, corresponding to approximately 18.9% of the dataset. The features `practice` and `caused` each contain 39 missing entries, while `drive` contains only one missing value. Overall, 102 rows contain at least one missing value, while 433 rows are complete.

Feature	Missing values	Missing percentage
complexity	101	18.88%
practice	39	7.29%
caused	39	7.29%
drive	1	0.19%

Table 2 – Semantic features with missing values in the cleaned dataset.

We decide not to remove rows with missing values at this stage. This choice is motivated by two reasons. First, the dataset is relatively small, and deleting all incomplete rows would reduce the number of available observations from 535 to 433. Second, because the target distribution is already highly imbalanced, removing observations could further weaken the representation of the minority classes. We also decide not to remove the four features with missing values, because the missingness is concentrated and manageable, and because the 65 attributes define the original semantic representation that we want to preserve. Missing values are therefore retained after the initial cleaning step and handled later through imputation. After structural cleaning, we create a train–test split. The split is performed before imputation and standardization, because all preprocessing parameters must be estimated only on the training data. We use an 80/20 split with stratification on `target_word_class`. Stratification is necessary because the target classes are strongly unbalanced, and a purely random split could produce unstable or poorly representative minority-class partitions. The resulting training set contains 428 observations, while the test set contains 107 observations. As reported in Table 3, the class proportions are preserved very closely across the full dataset, training set, and test set.

Split	Class	Count	Percentage
Full dataset	noun	434	81.12%
Full dataset	verb	62	11.59%
Full dataset	adjective	39	7.29%
Train	noun	347	81.07%
Train	verb	50	11.68%
Train	adjective	31	7.24%
Test	noun	87	81.31%
Test	verb	12	11.21%
Test	adjective	8	7.48%

Table 3 – Class distribution after stratified train–test splitting.

The stratified split also allows us to compute a first reference baseline. Since the majority class is *noun*, a classifier that always predicts *noun* would obtain an accuracy of approximately 0.811 on the full dataset. This value is not used as a final modelling result, but it is important as a warning: any supervised model must be evaluated against this trivial baseline and must be judged with metrics that account for minority-class performance. The third preprocessing step consists of missing-value imputation and feature standardization. Both operations are fitted exclusively on the training set and then applied to the test set. This is a central methodological constraint: if imputation values or scaling parameters were computed using the whole dataset, information from the test set would enter the training pipeline, producing data leakage and an overly optimistic evaluation**james2013introduction**. For imputation, we use median imputation. The median is preferred because it is simple, reproducible, and more robust than the mean in the presence of asymmetry or outlying values. Since the variables are numeric semantic ratings, median imputation provides a conservative replacement strategy that preserves the scale of each feature without introducing model-based assumptions. The imputer is fitted on the 428 training

observations. The median values learned from the training set are then used to impute both training and test data. As summarized in Table 4, before imputation, 81 training rows and 21 test rows contain at least one missing value. After imputation, both sets contain zero missing values.

Split	Rows with missing values before imputation	Rows with missing values after imputation
Train	81	0
Test	21	0

Table 4 – Effect of median imputation on missing values.

After imputation, we standardize all 65 semantic features using a standard score transformation. The scaler is fitted only on the imputed training set, estimating one mean and one standard deviation for each feature. These parameters are then applied to both training and test data. Standardization is required because several subsequent methods are scale-sensitive: PCA depends on variances and covariances, clustering and k-nearest-neighbor methods depend on distances, and regularized models such as Ridge, LASSO, and Elastic Net penalize coefficients in a way that is not invariant to predictor scale. Without standardization, features with larger numerical dispersion could dominate the analysis for purely metric reasons. The standardization diagnostics confirm that the transformation behaves as expected on the training data. After scaling, the maximum absolute feature mean in the training set is approximately 2.45×10^{-16}, while the feature standard deviations are numerically equal to one, up to floating-point precision. This confirms that the standardized training matrix is centered and scaled correctly. The test set is transformed using the training-set parameters, preserving its role as external evaluation data.

At the end of preprocessing, we obtain four main datasets:

1. the raw train and test sets,
2. the imputed but unscaled datasets,
3. the imputed and scaled train and test sets,
4. and the standardized train and test sets.

The imputed but unscaled datasets are useful for methods that do not require standardized predictors, whereas the standardized datasets are later used for PCA, clustering, distance-based methods, and regularized classification. The preprocessing phase therefore produces a controlled and leakage-free foundation for all subsequent analyses.

2.2 II. Unsupervised exploration of the semantic space

Before moving to supervised classification, we carry out an unsupervised exploration of the standardised training set. The goal of this phase is diagnostic rather than predictive: following (James et al., 2013), unsupervised learning is used here as a form of exploratory data analysis, aimed at assessing whether the observations organise themselves into meaningful groups independently of the known class labels(James et al., 2013). Two complementary tools are applied: Principal Component Analysis (PCA) for dimensionality reduction and visualisation, and clustering to search for natural groupings.

Principal Component Analysis

PCA was applied to the standardised feature matrix $\mathbf{X} \in \mathbb{R}^{428\times65}$, where each row represents a lexical item and each of the 65 columns one semantic feature. Standardisation to zero mean and unit variance, performed in Phase 1, is a prerequisite for PCA, since otherwise features measured on different scales would dominate the principal directions(James et al., 2013).

Variance explained. We examine the proportion of variance explained (PVE) by each principal component and the cumulative PVE (James et al., 2013). Figure 1 shows the scree plot; Figure 2 shows the cumulative curve. Retaining the first **10 principal components** accounts for approximately **75%** of the total variance. The first two components alone explain roughly **36%** of the total variance ($\text{cPVE}_2 \approx 0.36$), consistent with the high-dimensional and noisy nature of the semantic rating space. In accordance with the recommendation in (James et al., 2013), the number of components to retain was chosen by inspecting the elbow of the scree plot and selecting the smallest number that captures a substantial fraction of the total variation.

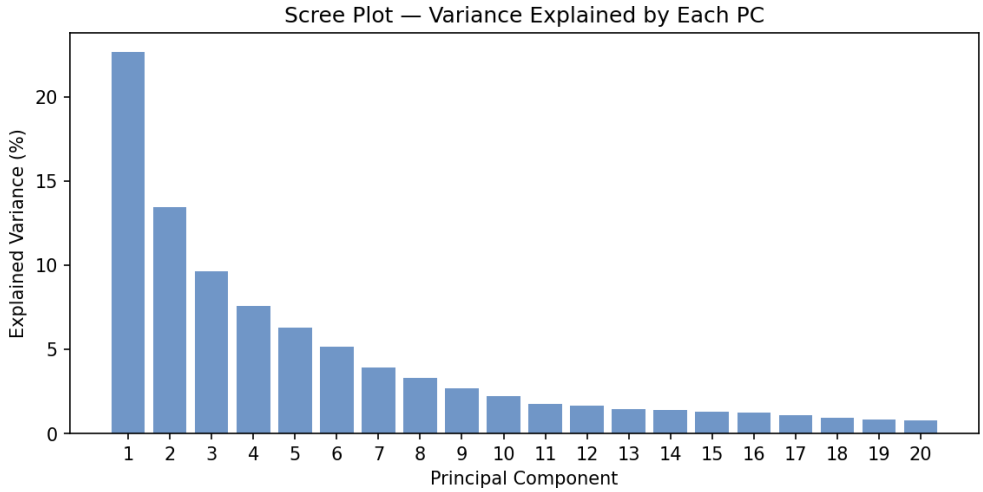


Figure 1 – Scree plot: proportion of total variance explained by each principal component. The contribution decreases steeply after the first few components.

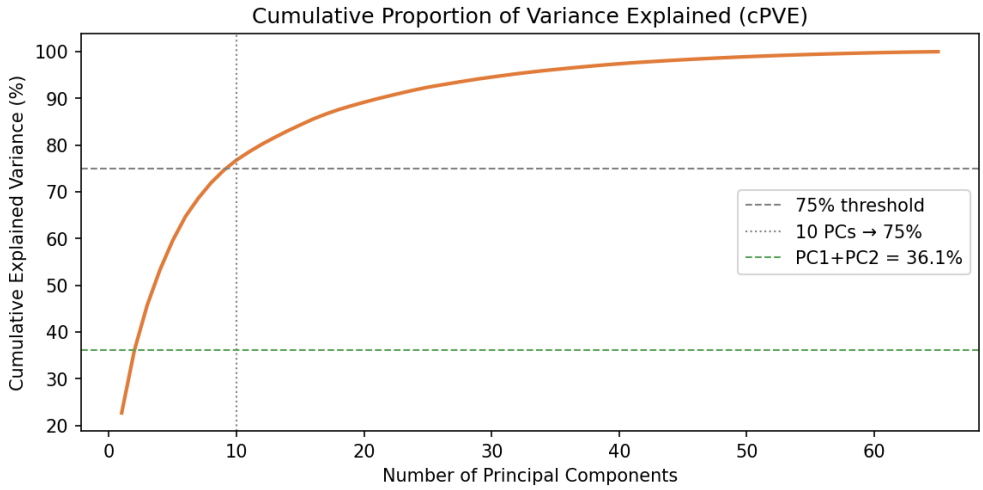


Figure 2 – Cumulative proportion of variance explained (cPVE). The dashed horizontal line marks the 75% threshold, reached at approximately 10 components.

Projection onto the first two components. Figure 3 shows the 428 training observations projected onto the plane defined by PC1 and PC2, coloured by morphosyntactic class. Following the biplot framework described in (James et al., 2013), the loading vectors reveal the semantic interpretation of each axis. PC1 is dominated by sensorimotor features (Vision, Colour, Shape, Touch, Texture), tracing a *concreteness* gradient along which nouns (*Thing*) cluster toward the concrete pole. PC2 captures auditory and affective features (Fast, Sound, Pain, Attention, Loud), separating more dynamic or eventive concepts from static ones. A broad but imperfect separation between word classes is visible, suggesting that the feature

space encodes class-relevant variation, though the boundaries are not sharp.

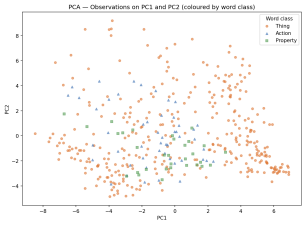


Figure 3 – Training observations projected onto PC1 and PC2. Colours indicate morphosyntactic class: *Thing* (nouns), *Action* (verbs), *Property* (adjectives). A concreteness gradient is visible along PC1.

Clustering

Clustering was performed on the PCA-reduced space retaining the first 10 principal components (those accounting for approximately 75% of the variance). Operating in this reduced space attenuates the effect of noise and high dimensionality on distance computations(James et al., 2013), which notes that many statistical techniques, including clustering, can benefit from using the first few principal component score vectors rather than the full feature matrix.

Two methods were applied: K-means clustering and Agglomerative Hierarchical Clustering (AHC) with complete linkage and Euclidean distance, both introduced in (James et al., 2013).

Selecting the number of clusters. The optimal K was investigated using two criteria.

1. **Elbow method** (K-means): the within-cluster inertia was plotted as a function of K (Figure 4). As noted in (James et al., 2013), minimising the within-cluster variation is the objective of K-means; the elbow in the inertia curve indicates where additional clusters yield diminishing returns. The Hartigan index, defined as $H(K) = \big(\text{inertia}(K)/\text{inertia}(K{+}1) - 1\big)(n - K - 1)$, formalises this criterion; both point to $K = 2$ as the most parsimonious solution.
2. **Average silhouette score**: the mean silhouette coefficient was computed for $K \in \{2, \ldots, 15\}$ for both methods (Figures 5 and 6). The silhouette scores are modest throughout, reflecting the absence of strongly separated clusters in this data. The highest values are obtained at small K, with $K = 2$ yielding approximately 0.43 for K-means and 0.16 for AHC.

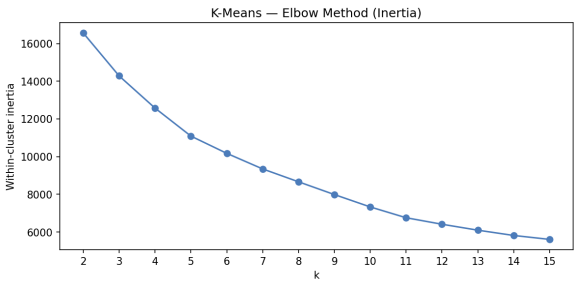


Figure 4 – K-means: within-cluster inertia (elbow method).

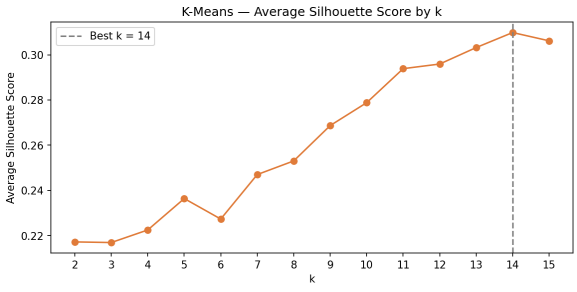


Figure 5 – K-means: average silhouette score by K.

Given the flat silhouette profile, two values of K are considered in the visualisations: $K = 2$ (indicated by both criteria) and $K = 10$ (a semantically richer partition). An important caveat noted in (James et al., 2013) is that K-means finds a *local* rather than global optimum; for this reason the algorithm was run with 20 random initialisations and the solution minimising the objective was retained.

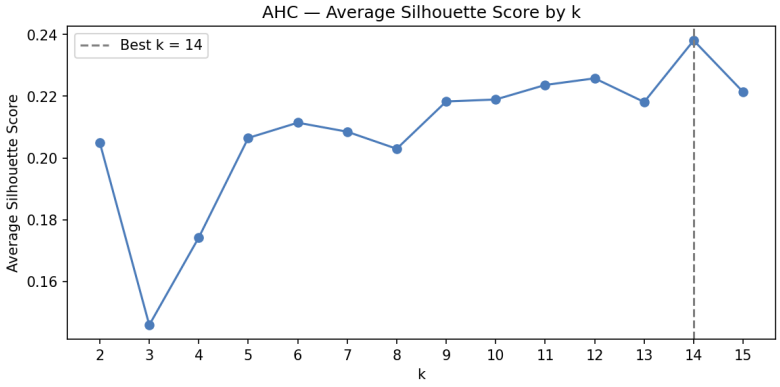


Figure 6 – AHC (complete linkage, Euclidean distance): average silhouette score by K.

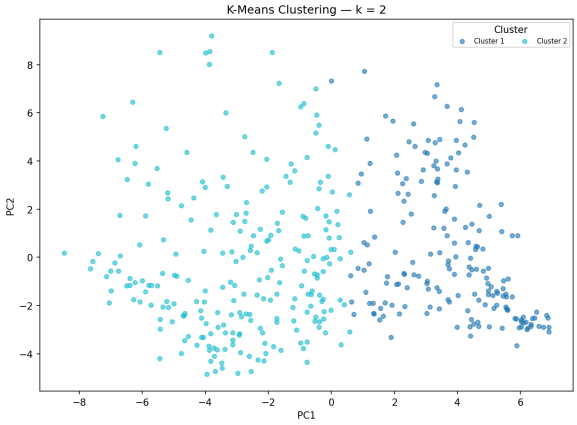


Figure 7 – K-means, $K = 2$: the partition roughly follows the concreteness gradient of PC1.

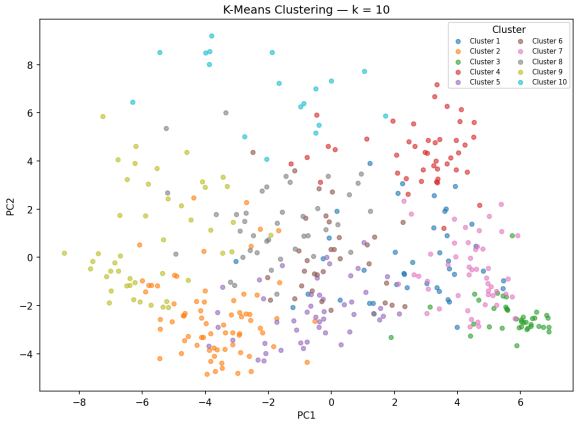


Figure 8 – K-means, $K = 10$: finer-grained semantic domains begin to emerge.

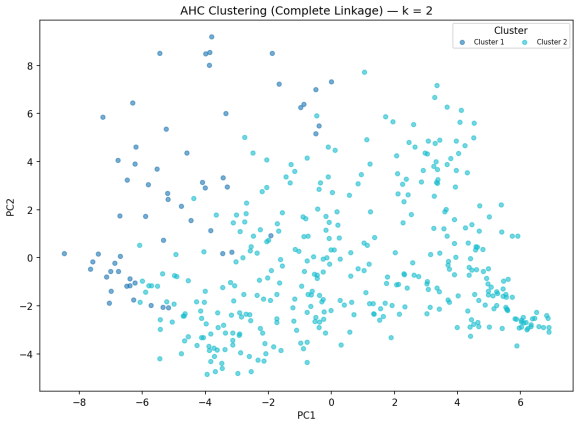


Figure 9 – AHC, $K = 2$.

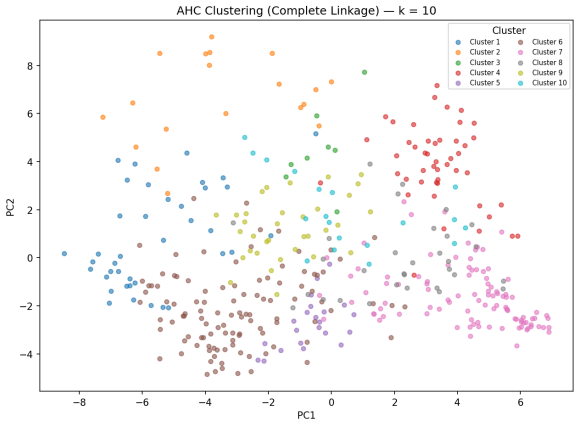


Figure 10 – AHC, $K = 10$: qualitatively more coherent semantic groupings than K-means (e.g. animals, food, negative affect).

Comparison with known labels

To assess the degree of correspondence between the data-driven cluster structure and the three morphosyntactic classes (*Thing*, *Action*, *Property*), the cluster assignments at $K = 3$ were compared with the ground-truth labels using two external validity indices: the Adjusted Rand Index (ARI) and the Normalised Mutual Information (NMI). These metrics are not introduced in (James et al., 2013) but are standard in the clustering evaluation literature; ARI equals 1 for a perfect match and is 0 in expectation for a random partition.

Table 5 reports ARI and NMI for both methods at $K = 3$.

Table 5 – External validity of cluster assignments at $K = 3$ compared with the three morphosyntactic labels. Values to be filled after running `phase2_unsupervised.py` and reading from `outputs/II/phase2_unsupervised/phase2_summary.json`.

Method	ARI	NMI
K-means ($K = 3$)	–	–
AHC ($K = 3$)	–	–

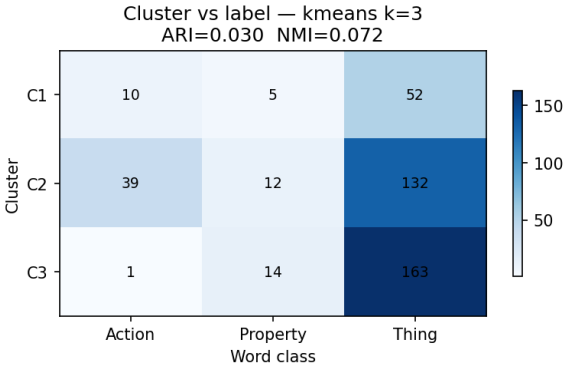


Figure 11 – Cluster–label correspondence heatmap, K-means $K = 3$.

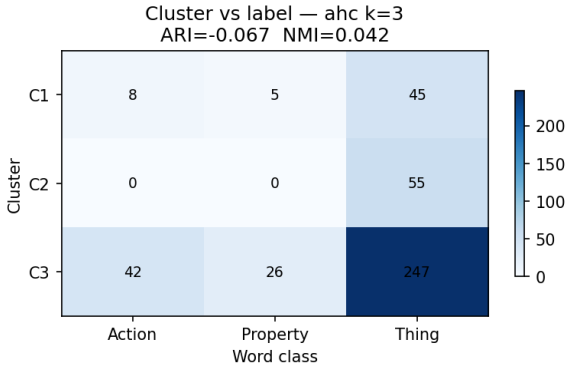


Figure 12 – Cluster–label correspondence heatmap, AHC $K = 3$.

Interpretation

The unsupervised analysis yields three main findings.

First, the data-driven cluster structure does *not* align strongly with the morphosyntactic partition. This is consistent with the intrinsic difficulty of the task: the three word classes are not cleanly separable in the semantic feature space, and the dataset is strongly imbalanced (nouns account for approximately 81% of the training set).

Second, at $K = 2$, both methods identify a broad **concrete vs. abstract** dimension that cuts across morphosyntactic boundaries. The concrete cluster is dominated by nouns referring to physical entities (e.g. *elephant*, *bread*), but also contains action verbs denoting physical events. The abstract cluster groups most verbs and adjectives together with nouns referring to mental or social concepts. This concrete–abstract axis corresponds to the first principal component and is the most prominent structure in the feature space.

Third, at $K = 10$, AHC with complete linkage recovers several semantically coherent groups — including clusters of animal names, food items, sounds, and negatively valenced concepts — that correspond to fine-grained *semantic categories* rather than to the grammatical classes of interest. This confirms that

the 65-dimensional feature space encodes rich semantic similarity information at a level of granularity finer than the noun/verb/adjective distinction, and motivates the use of a supervised approach for the classification task.

These results serve a diagnostic function in the overall pipeline: they confirm the semantic informativeness of the feature space while showing that the class labels are not trivially recoverable from unsupervised exploration alone.

2.3 III. Supervised classification on the original feature space

In this phase, we formulate a multiclass supervised learning problem aimed at predicting the lexical-semantic labels from the original semantic feature space. The objective is to quantify how much of the categorical structure of lexical meaning is captured by the experiential attributes and to understand the role of redundancy, multicollinearity, and model complexity in this setting.

Baseline model We begin with a multinomial logistic regression model, which provides a natural baseline within the generalized linear model framework for multiclass classification, modelling class log-odds as functions of the predictors **james2013introduction**; (Hastie et al., 2009). This model assumes a linear relationship between the predictors and the log-odds of class membership and serves as a reference point for all subsequent analyses. The baseline achieves strong performance, as reported in Table 6.

Model	Accuracy	Macro F1
Multinomial Logistic baseline	0.9533	0.8904

Table 6 – Baseline multinomial logistic regression performance on the original labels.

The high accuracy indicates that the semantic feature space is highly informative. However, the lower macro F1 (0.8904) compared to accuracy suggests an imbalance in class-wise performance, meaning that some classes are easier to predict than others. This behavior is confirmed by the confusion matrix in Figure 13. From the confusion matrix, it is evident that misclassifications are not uniformly distributed across classes, indicating overlapping regions in the feature space.

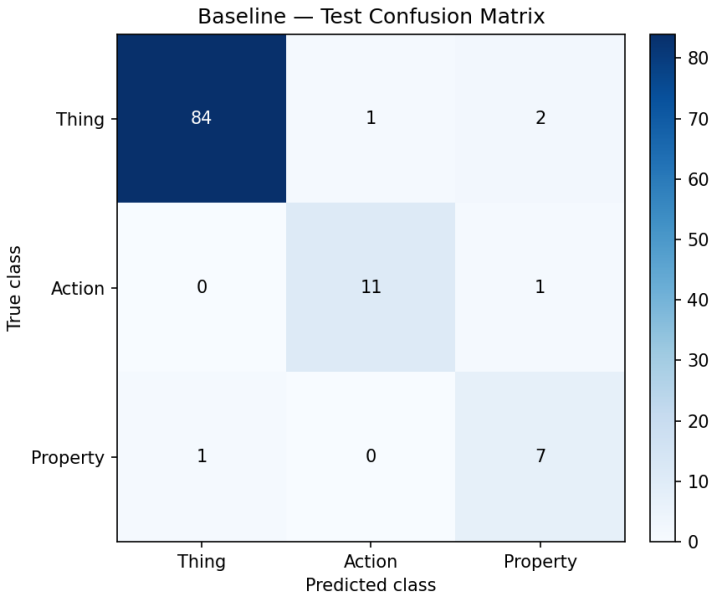


Figure 13 – Confusion matrix of the baseline multinomial logistic regression.

Multicollinearity and feature structure Before applying more advanced models, we analyze the structure of the feature space. The correlation heatmap in Figure 14 shows the presence of strongly correlated blocks of variables.

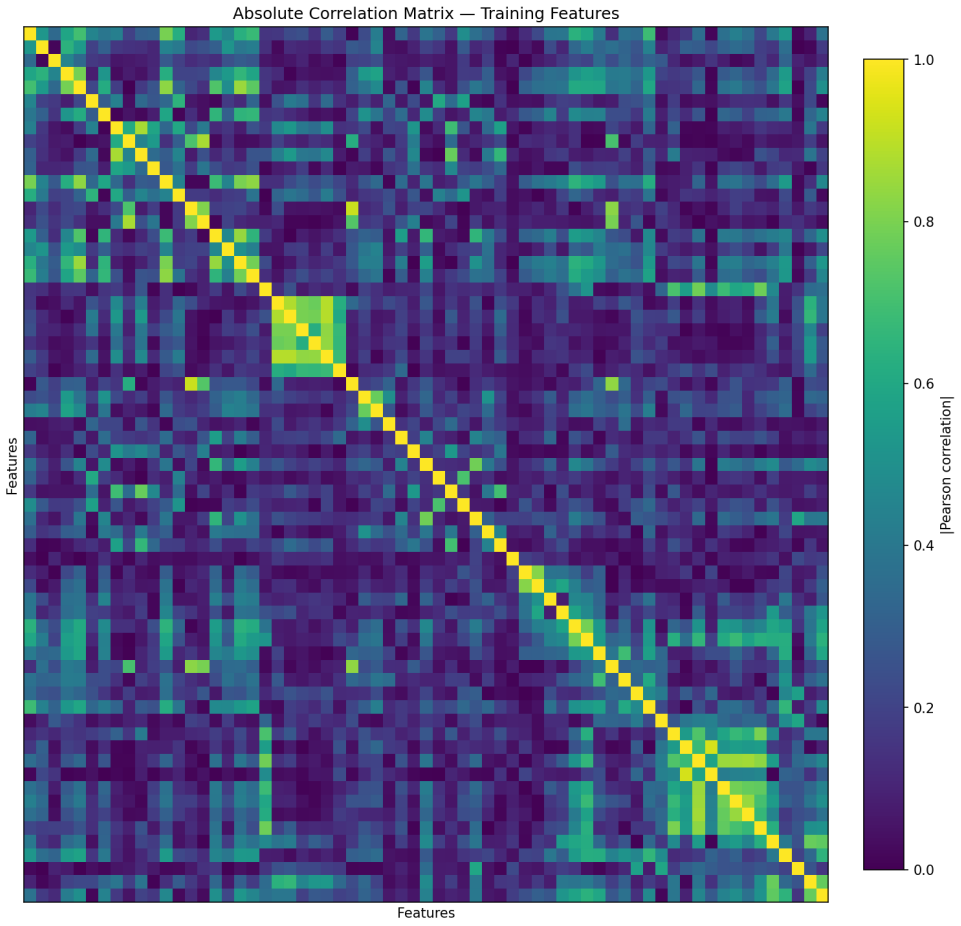


Figure 14 – Correlation heatmap of semantic features.

This structure is further confirmed by the hierarchical clustering of features (Figure 15), which groups variables into coherent clusters.

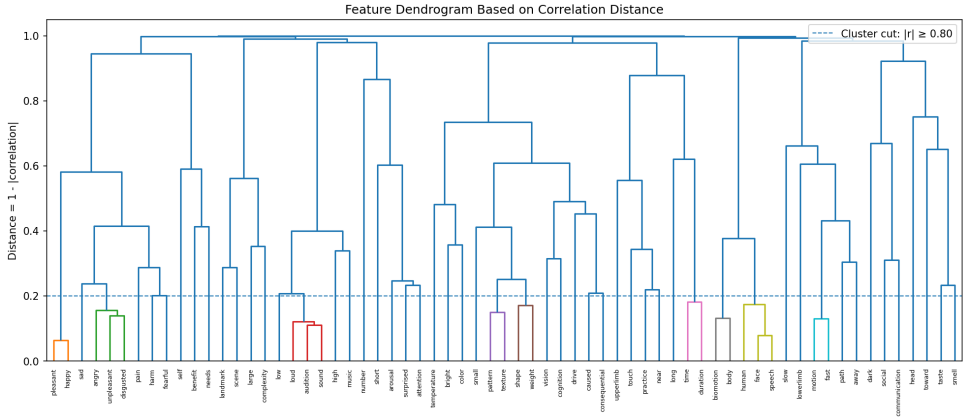


Figure 15 – Hierarchical clustering dendrogram of features.

Numerically, this is reflected in the high number of strongly correlated pairs (22 pairs with $|r| \geq 0.8$) and

in the presence of many variables with high VIF values (51 features with VIF ≥ 5). These values confirm that the feature space is highly redundant and that naive models may suffer from instability.

Regularized models To address multicollinearity and improve generalization, we apply LASSO and Elastic Net regularized multinomial logistic regression models.

Model	Accuracy	Macro F1
LASSO Logistic	0.9680	0.9150
Elastic Net Logistic	0.9705	0.9200

Table 7 – Performance of regularized multinomial models.

Both models improve over the baseline, with Elastic Net achieving the best balance between accuracy and macro F1. The improvement in macro F1 (from 0.8904 to 0.9200) is particularly important, as it indicates a more balanced classification across all classes.

Sparsity and feature selection Regularization also induces sparsity, reducing the number of active predictors. The most relevant features selected by LASSO are shown in Figure 16.

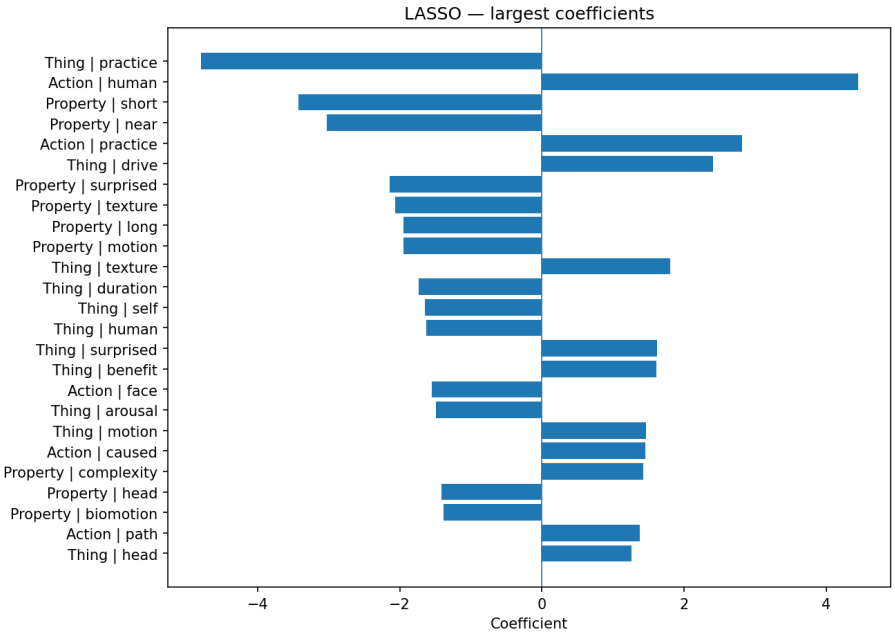


Figure 16 – Top LASSO coefficients.

Elastic Net, shown in Figure 17, distributes weights more smoothly across correlated variables, consistently with its tendency to retain groups of related predictors more stably than pure LASSO in the presence of multicollinearity (Zou et Hastie, 2005).

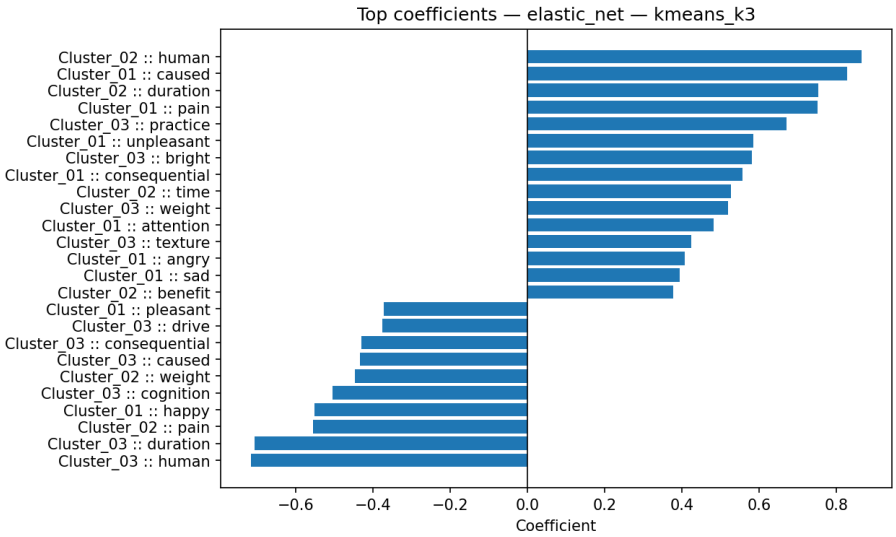


Figure 17 – Top Elastic Net coefficients.

The sparsity overview in Figure 18 shows how many coefficients are effectively used by the models.

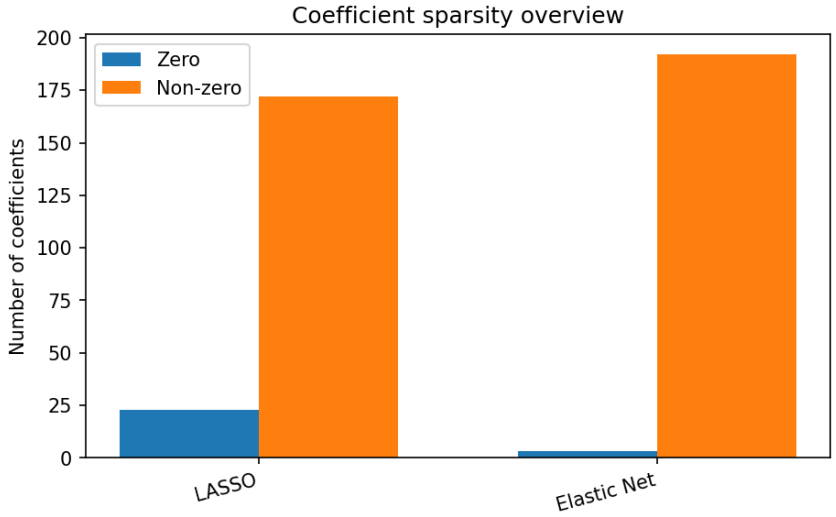


Figure 18 – Sparsity patterns across models.

These results indicate that only a subset of the semantic features is necessary to achieve high predictive performance, confirming the presence of redundancy in the original representation.

Final post-selection model After feature selection, we train a final multinomial logistic model on the selected subset.

Model	Accuracy	Macro F1
Post-selection Logistic	0.9720	0.9229

Table 8 – Final post-selection model.

The improvement from 0.9533 to 0.9720 in accuracy and from 0.8904 to 0.9229 in macro F1 demonstrates that removing redundant features improves generalization.

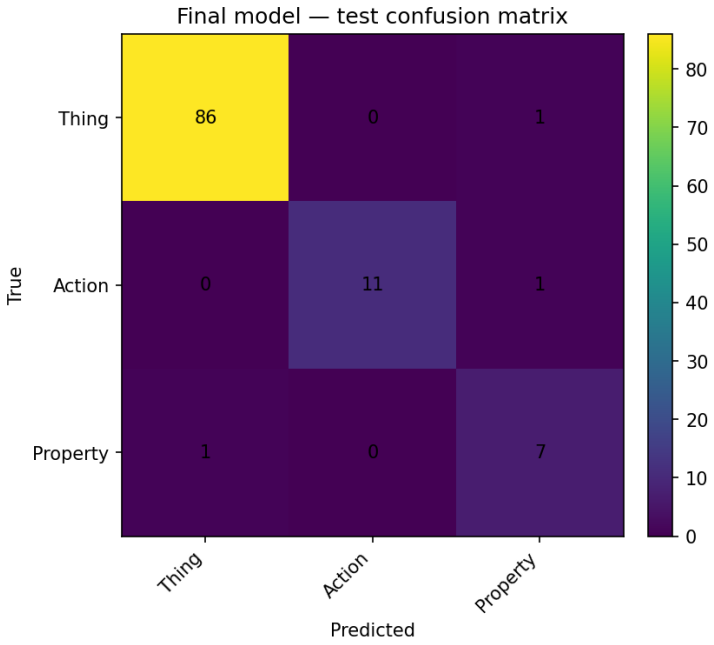


Figure 19 – Confusion matrix of the final model.

Discussion Overall, the results show that:

— the semantic feature space is highly informative;
— multicollinearity is substantial and must be controlled;
— regularization improves both stability and performance;
— a reduced subset of features is sufficient for accurate classification.

2.4 IV. Alternative classifiers

To assess whether the classification results depend specifically on the multinomial logistic framework, an additional comparison was performed using alternative classifiers on the same selected semantic feature space. These classifiers represent complementary modelling assumptions: SVMs search for maximum-margin decision boundaries (Cortes et Vapnik, 1995), Random Forests aggregate multiple decorrelated decision trees (Breiman, 2001), and k-NN implements a local distance-based classification rule (Cover et Hart, 1967). The comparison is therefore designed to isolate the effect of the classifier while keeping the data representation fixed. The input matrix used in this phase contains 65 selected semantic predictors, with 428 training observations and 107 test observations. The target labels are the operational word-class labels used in the scripts: *adjective*, *noun*, and *verb*. The class distribution remains highly imbalanced, with nouns representing about 81% of both training and test data. For this reason, the results are evaluated not only through accuracy, but also through balanced accuracy, macro-F1, and weighted-F1.

Split	Class	Count	Percentage
Train	adjective	31	7.24%
Train	noun	347	81.07%
Train	verb	50	11.68%
Test	adjective	8	7.48%
Test	noun	87	81.31%
Test	verb	12	11.21%

Table 9 – Class distribution in the alternative-classifier comparison.

Table 10 reports the test-set performance of all classifiers. The dummy majority classifier reaches an accuracy of 0.813, which is superficially high because nouns dominate the test set. However, its balanced accuracy is only 0.333 and its macro-F1 is 0.299, showing that it completely fails on the minority classes. This confirms that accuracy alone is not an adequate metric for this dataset.

Classifier	**Accuracy**	**Balanced accuracy**	**Macro-F1**	**Weighted-F1**
Random Forest	0.981	0.917	0.949	0.980
SVM linear	0.972	0.989	0.942	0.974
SVM RBF	0.972	0.927	0.938	0.972
k-NN	0.953	0.819	0.883	0.950
Post-selection Logistic	0.944	0.915	0.875	0.947
Dummy majority	0.813	0.333	0.299	0.729

Table 10 – Test performance of the alternative classifiers on the selected semantic feature space.

The best overall model by macro-F1 is the Random Forest, with a test macro-F1 of 0.949 and an accuracy of 0.981. The linear SVM follows closely, with macro-F1 0.942, but it achieves the highest balanced accuracy, 0.989, indicating the best average recall across the three classes. The RBF SVM performs similarly, with accuracy 0.972 and macro-F1 0.938. The post-selection logistic model remains strong and interpretable, but is slightly below the SVM and Random Forest classifiers. Finally, k-NN obtains good raw accuracy, 0.953, but lower balanced accuracy, 0.819, indicating weaker behaviour on at least one minority class. This is plausible because distance-based classifiers are sensitive to class imbalance and to the geometry of the feature space, especially when minority classes occupy small or sparse regions (Cover et Hart, 1967); (He et Garcia, 2009).

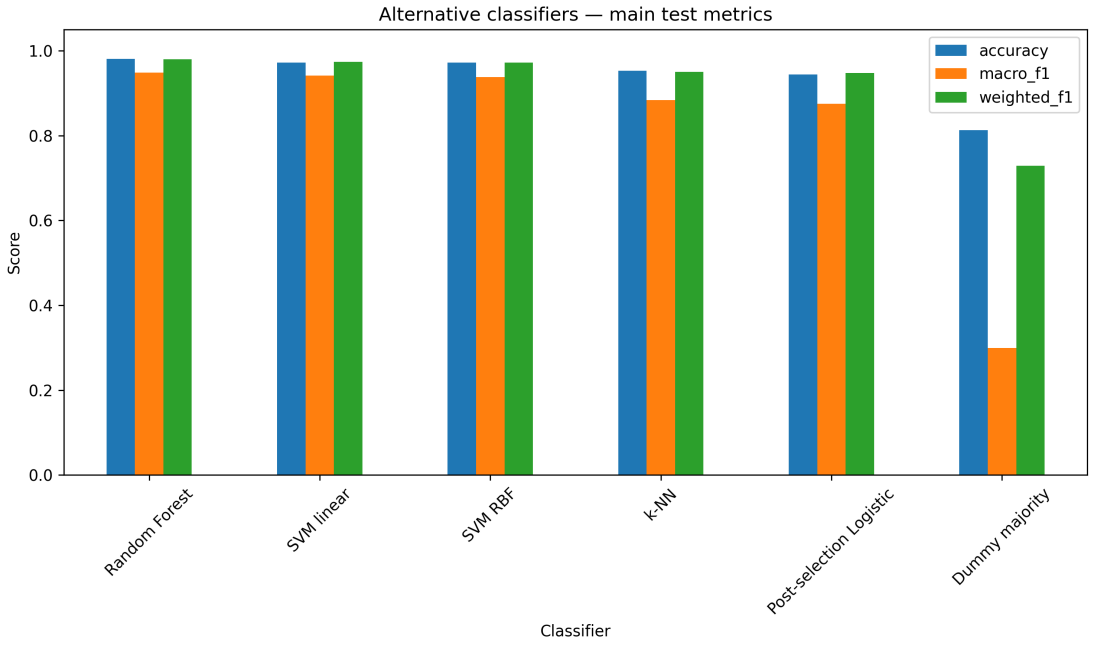


Figure 20 – Main test metrics for the alternative classifiers.

Figure 20 shows that all substantive classifiers clearly outperform the dummy majority baseline. The gap is especially visible for macro-F1, which penalizes classifiers that ignore minority classes. Weighted-F1 is systematically higher than macro-F1 because it is dominated by the majority noun class.

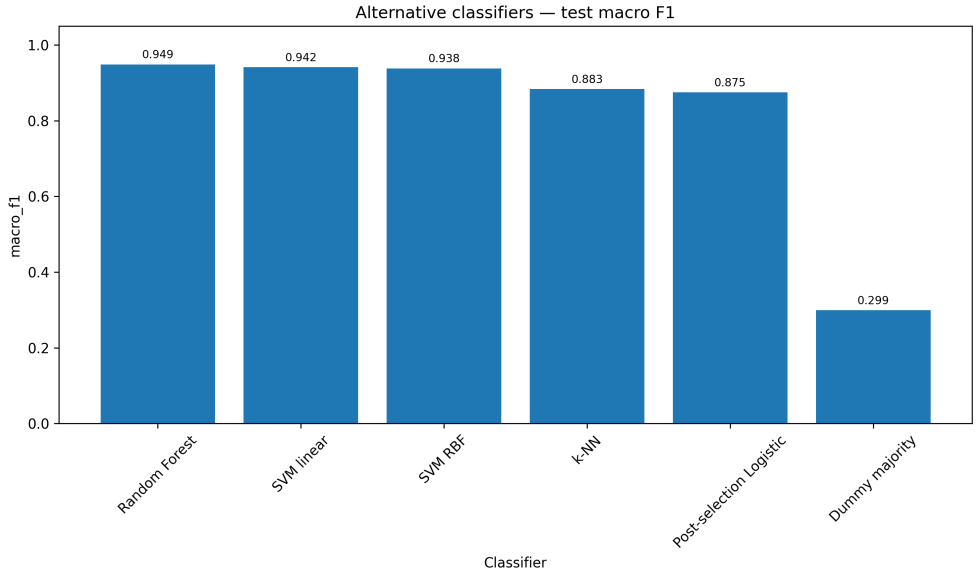


Figure 21 – Macro-F1 comparison across alternative classifiers.

The macro-F1 comparison in Figure 21 confirms the ranking of the classifiers. Random Forest achieves the best balance between precision and recall across classes, followed by the two SVM variants. The post-selection logistic model remains competitive, but the tree ensemble and margin-based classifiers exploit the selected semantic feature space more effectively.

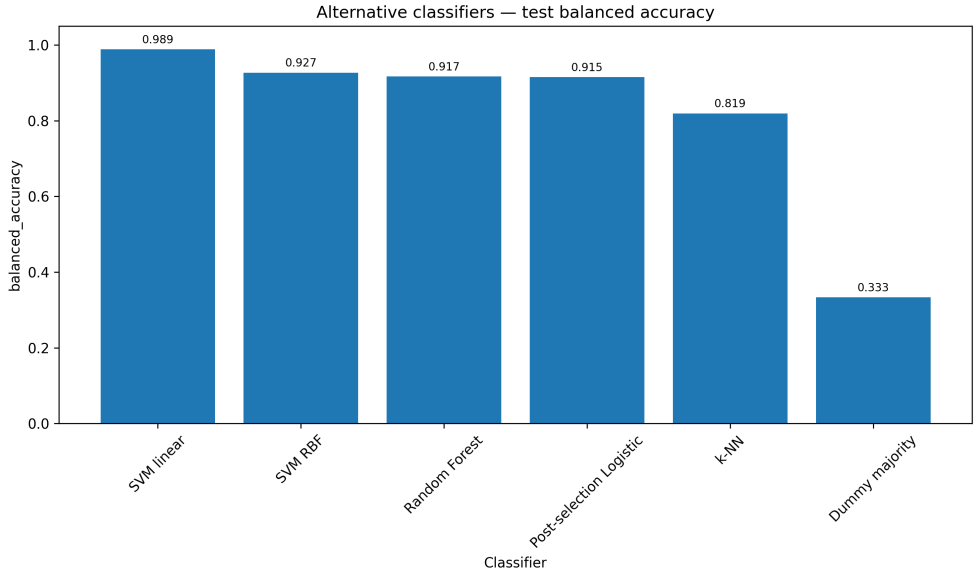


Figure 22 – Balanced accuracy comparison across alternative classifiers.

Balanced accuracy, shown in Figure 22, gives a slightly different picture. The linear SVM is the best model under this metric, because it correctly recovers all adjectives and all verbs, while making only three errors on nouns. Random Forest has the best macro-F1 and accuracy, but its balanced accuracy is lower because it misses two adjectives.

The confusion matrices provide a more detailed interpretation of these results. The dummy majority classifier assigns every observation to *noun*; therefore, it correctly classifies all nouns but fails completely on adjectives and verbs. The post-selection logistic model improves substantially, correctly classifying

7/8 adjectives, 83/87 nouns, and 11/12 verbs. The linear SVM correctly classifies all adjectives and all verbs, with only three nouns predicted as adjectives. The RBF SVM makes only three errors overall. Random Forest makes only two errors, both adjectives predicted as nouns. The k-NN classifier correctly classifies all nouns, but misclassifies three adjectives and two verbs as nouns, which explains its lower balanced accuracy.

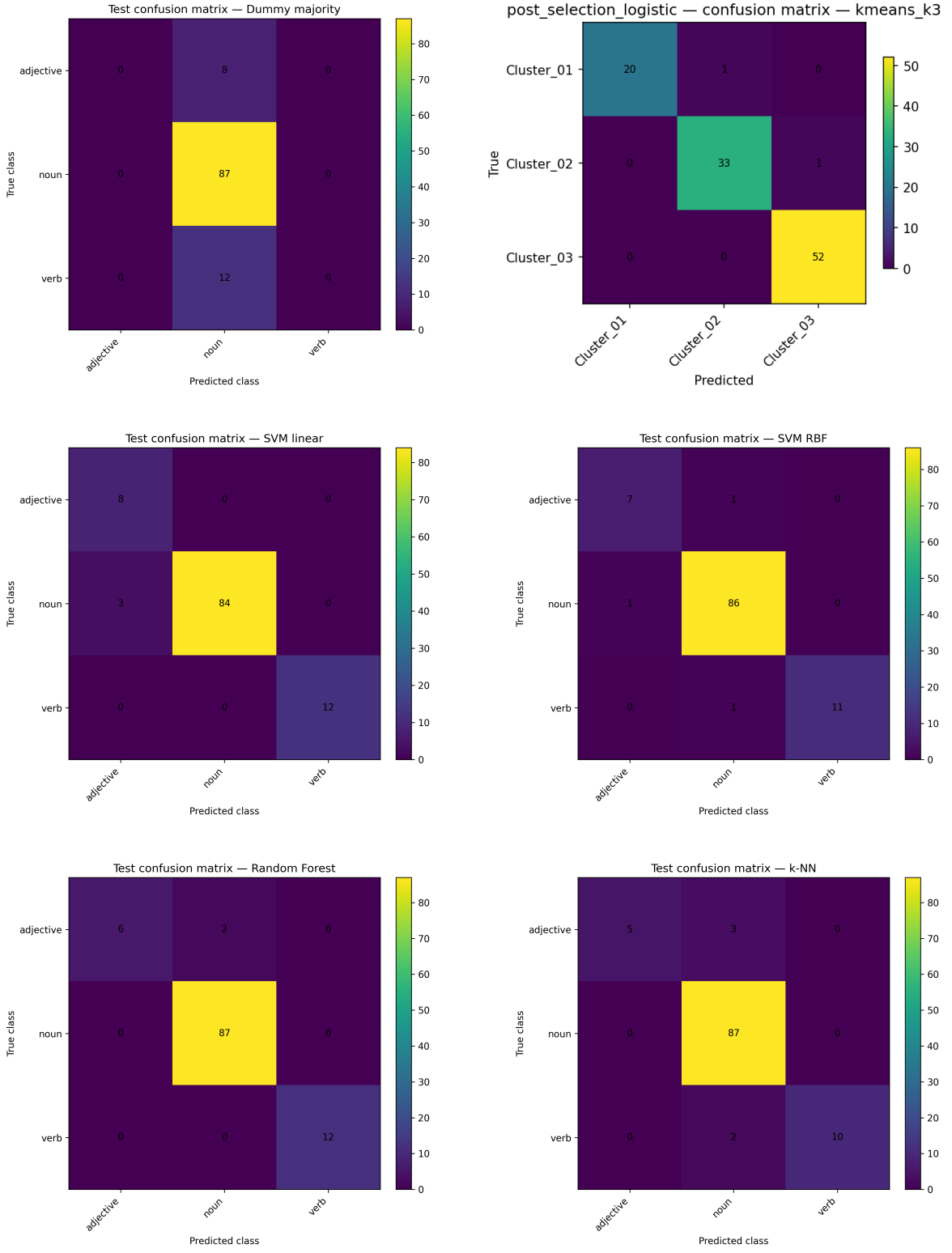


Figure 23 – Test confusion matrices for the alternative classifiers. From top-left to bottom-right: dummy majority, post-selection logistic, linear SVM, RBF SVM, Random Forest, and k-NN.

The hyperparameter search was performed using three-fold cross-validation with macro-F1 as the scoring criterion. Table 11 reports the selected configurations. The RBF SVM obtains the best cross-validated macro-F1, while Random Forest obtains the best test macro-F1. This difference suggests that the final test ranking should be interpreted cautiously, given the small number of minority-class observations in the test set.

Classifier	**Best parameters**	**Best CV macro-F1**
Post-selection Logistic	$C = 1.0$	0.904
SVM linear	$C = 1.0$	0.890
SVM RBF	$C = 1.0,\ \gamma = 0.01$	0.915
Random Forest	$n_{\text{estimators}} = 100,$ max_depth = `None`, min_samples_split = 2	0.868
k-NN	$k = 3,$ metric = `euclidean`, weights = `uniform`	0.844

Table 11 – Best hyperparameters selected by cross-validation for the alternative classifiers.

Overall, the comparison shows that the selected semantic feature space is highly predictive of word class across different classifier families. The dummy majority baseline demonstrates the misleading nature of raw accuracy under class imbalance. The post-selection logistic model provides a strong and interpretable reference point, while SVMachieves the best balanced accuracy and the strongest minority-class recall, suggesting that the selected semantic space is close to being linearly separable under a maximum-margin criterion (Cortes et Vapnik, 1995), and Random Forest achieves the best global accuracy and macro-F1, consistently with the ability of tree ensembles to capture nonlinear effects and interactions among predictors without specifying them explicitly in a parametric model (Breiman, 2001). This indicates that the semantic predictors selected in the previous phases contain robust class-discriminative information, not tied to a single modelling framework.

2.5 V. High-dimensional expansion and feature screening

After the first supervised analyses on the original semantic representation, the project moves to a high-dimensional version of the Binder/Testa dataset. The purpose of this phase is to test whether the three lexical-semantic classes can be better separated when the original semantic attributes are enriched with nonlinear and interaction-based information. The starting point is the standardized matrix of 65 primitive semantic predictors. For each lexical item, let

$$X = (X_1, X_2, \ldots, X_{65})$$

be the vector of semantic attributes. The high-dimensional representation is obtained by augmenting this vector with all second-degree terms. More precisely, the expanded matrix contains the original semantic features, the squared terms, and all pairwise interactions:

$$X_1, \ldots, X_{65},$$

$$X_1^2, \ldots, X_{65}^2,$$

$$X_j X_k, \qquad j < k.$$

The number of pairwise interactions is

$$\binom{65}{2} = \frac{65 \cdot 64}{2} = 2080.$$

Therefore, the total number of predictors in the high-dimensional dataset is

$$65 + 65 + 2080 = 2210.$$

The transformation changes the structure of the supervised problem. The original dataset has more observations than predictors, whereas the expanded representation has more predictors than training observations. After the train-test split, the training set contains 428 observations and 2210 predictors, while the test set contains 107 observations and the same 2210 predictors. The dimensionality ratio is therefore

$$\frac{p}{n_{\text{train}}} = \frac{2210}{428} \approx 5.16.$$

This means that the number of predictors is more than five times larger than the number of training observations. Table 12 reports the numerical structure of the expanded feature space.

Quantity	**Value**
Training observations	428
Test observations	107
Original semantic predictors	65
Squared terms	65
Pairwise interaction terms	2080
Total expanded predictors	2210
Ratio p/n_{train}	5.16

Table 12 – Numerical structure of the high-dimensional semantic feature space.

The expansion has a semantic motivation. The original predictors measure the marginal contribution of individual experiential dimensions, such as `vision`, `motion`, `practice`, `social`, `cognition`, or `emotion`. The squared terms allow the model to capture nonlinear effects of single semantic dimensions. The interaction terms, instead, represent combined semantic profiles. For instance, an interaction such as `body_x_practice` may be informative for action-related lexical items, while terms such as `human_x_self`, `path_x_cognition`, or `practice_x_caused` may capture more complex semantic patterns involving agency, causality, action structure, and abstract conceptual organization.

At the same time, this expansion creates a statistically difficult learning problem. The original semantic dimensions are already naturally correlated, because experiential attributes belonging to related domains tend to co-occur in word meanings. The second-degree expansion amplifies this dependence: squared terms are directly derived from original variables, and interaction terms share components with one another. Consequently, the expanded matrix is highly redundant and necessarily rank-deficient. Since $p = 2210$ and $n_{\text{train}} = 428$, the rank of the training matrix cannot exceed 428. A lower bound for the rank deficiency is therefore

$$2210 - 428 = 1782.$$

This confirms that an ordinary unregularized multinomial model is not suitable in the expanded feature space. The matrix $X^\top X$ is singular, and classical coefficient estimates would be unstable or non-unique.

This is the reason why the high-dimensional phase requires collinearity diagnostics, screening, and regularized models.

The first diagnostic step investigates pairwise correlations among the 2210 expanded predictors. Figure 24 shows the distribution of absolute pairwise correlations in the high-dimensional training matrix. Most feature pairs have low or moderate correlations, but the long right tail indicates the presence of many strongly correlated pairs. This is expected, since several expanded terms share one original semantic component or combine semantically related dimensions.

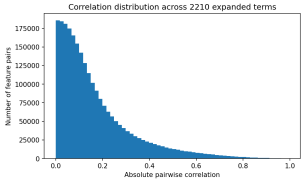


Figure 24 – Distribution of absolute pairwise correlations across the 2210 expanded predictors.

The extent of multicollinearity is summarized in Table 13. A large number of feature pairs exceed standard correlation thresholds: more than 127000 pairs have absolute correlation above 0.50, almost 29000 exceed 0.70, and 1945 pairs exceed 0.90. These results confirm that the high-dimensional expansion does not simply increase the number of variables, but also creates a dense structure of redundant and partially overlapping predictors.

Absolute correlation threshold	**Number of feature pairs**
≥ 0.50	127467
≥ 0.70	28737
≥ 0.80	9962
≥ 0.90	1945
≥ 0.95	404
≥ 0.99	3

Table 13 – Number of highly correlated feature pairs in the expanded training matrix.

Figure 25 reports the expanded terms most involved in high-correlation relationships. Many of these terms belong to coherent semantic domains or combine closely related dimensions, such as auditory, facial, bodily, affective, and perceptual attributes. This confirms that multicollinearity is not random: it reflects the semantic organization of the original attributes and the algebraic dependencies introduced by the second-degree expansion.

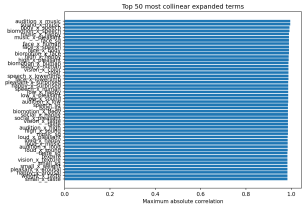


Figure 25 – Top expanded terms involved in high pairwise correlations.

Because of this structure, the next step is feature screening. The goal of screening is not to estimate the final classifier directly, but to reduce the high-dimensional space before fitting penalized multinomial models. In this phase, each predictor is evaluated marginally with respect to the target variable. This is

appropriate in the present setting because the number of predictors is much larger than the number of observations, and a direct search over all possible combinations of features would be statistically unstable and computationally inefficient.

The screening procedure uses three complementary criteria. The first is the ANOVA F-score, which measures how strongly each feature differs across the three lexical-semantic classes. The second is mutual information, which captures a more general association between each predictor and the target. The third is the marginal class-mean contrast, defined as the largest class-level separation produced by a feature. These criteria are then combined into a single ranking, from which the top 300 predictors are retained.

The reduction is substantial:

$$2210 \longrightarrow 300.$$

This type of preliminary reduction is standard in ultrahigh-dimensional supervised learning, where screening makes the subsequent modelling step statistically more stable and computationally feasible (Fan et Lv, 2008); (Fan et al., 2009); (Nandy et al., 2022). The screening step removes 1910 features, corresponding to approximately 86.43% of the expanded feature space. The retained set represents only 13.57% of the original 2210 predictors. Table 14 summarizes the screening phase.

Quantity	**Value**
Features before screening	2210
Features retained after screening	300
Features removed	1910
Retained percentage	13.57%
Removed percentage	86.43%

Table 14 – Summary of the feature screening step on the high-dimensional representation.

Figure 26 shows the top predictors according to the ANOVA F-score. The highest-ranked terms are almost entirely interaction features, especially combinations involving `practice`, `short`, `human`, `biomotion`, `body`, `path`, `duration`, `self`, and `cognition`. This suggests that class separation is strongly associated with joint semantic configurations rather than only with isolated primitive attributes.

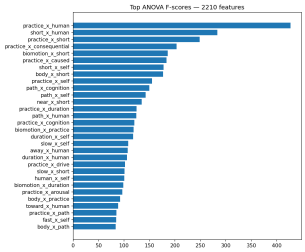


Figure 26 – Top ANOVA F-scores computed on the 2210-feature high-dimensional representation.

Figure 27 reports the top predictors according to mutual information. Compared with the ANOVA ranking, mutual information gives more prominence to both original and squared terms, such as `complexity`, `complexity_sq`, `practice`, `practice_sq`, `caused`, and `caused_sq`. This indicates that some individual semantic dimensions retain strong predictive value even without being combined with other features. At the same time, several interaction terms remain highly ranked, including `practice_x_caused`, `complexity_x_practice`, `complexity_x_caused`, `practice_x_human`, and `short_x_human`.

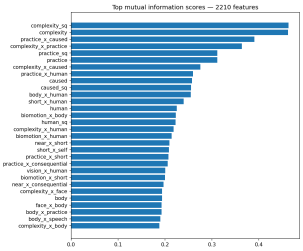


Figure 27 – Top mutual information scores computed on the 2210-feature high-dimensional representation.

Figure 28 displays the top marginal class-mean contrasts. This ranking is particularly useful for identifying predictors that produce strong separation between at least one lexical class and the others. Again, the top positions are dominated by interaction terms involving action-related, bodily, temporal, causal, and human-related dimensions. The strongest contrasts include `practice_x_human`, `short_x_human`, `practice_x_short`, `practice_x_consequential`, `biomotion_x_short`, `body_x_short`, and `short_x_self`.

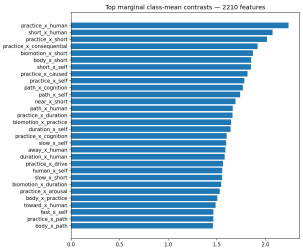


Figure 28 – Top marginal class-mean contrasts computed on the 2210-feature high-dimensional representation.

The final combined ranking is reported in Figure 29. This plot provides the actual set of highest-priority predictors selected by the screening procedure. The most relevant terms include `practice_x_human`, `short_x_human`, `practice_x_caused`, `practice_x_short`, `practice_x_consequential`, `biomotion_x_short`, `short_x_self`, `near_x_short`, `practice_x_self`, and `body_x_short`. The prevalence of interaction terms confirms that the high-dimensional expansion is informative: the strongest marginal predictors are not merely the original semantic attributes repeated in transformed form, but combinations of semantic dimensions that appear to encode discriminative lexical-class profiles.

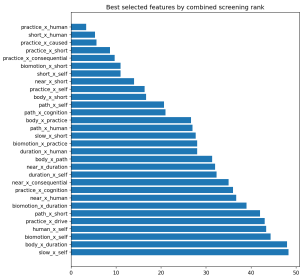


Figure 29 – Best selected features according to the combined screening rank.

Rank	Selected feature
1	practice_x_human
2	short_x_human
3	practice_x_caused
4	practice_x_short
5	practice_x_consequential
6	biomotion_x_short
7	short_x_self
8	near_x_short
9	practice_x_self
10	body_x_short
11	path_x_self
12	path_x_cognition
13	body_x_practice
14	path_x_human
15	slow_x_short

Table 15 – First fifteen predictors selected by the combined screening procedure.

From an interpretive point of view, the selected features are coherent with the lexical-semantic task. Many of the strongest terms involve practice, body, biomotion, and path, which are naturally related to action structure. Other terms involve human, self, cognition, caused, and consequential, which can help separate more abstract, relational, or property-like meanings. The presence of temporal and spatial components such as short, duration, near, and path suggests that lexical classes are not distinguished only by perceptual concreteness, but also by structured combinations of event-related and relational semantic dimensions.

The output of this phase is therefore a reduced high-dimensional matrix containing 300 screened predictors. This matrix preserves the most informative terms from the original 2210-feature expansion while removing the majority of weak or redundant predictors. It is used as the input for the following regularized multinomial models, where LASSO and Elastic Net further refine the feature set and estimate sparse class-discrimination patterns.

2.6 VI. Final model comparison and interpretation

This final phase integrates the results obtained across all previous analyses, with the objective of identifying the models that provide the best trade-off between predictive performance, statistical stability, and interpretability. The comparison includes the baseline multinomial logistic regression, the regularized models (LASSO and Elastic Net), the post-selection logistic model, and the alternative non-linear classifiers. From a purely predictive perspective, the results clearly indicate that performance improves progressively along the pipeline. The baseline multinomial logistic model achieves an accuracy of 0.9533 and a macro F1-score of 0.8904. Regularization leads to consistent improvements, with Elastic Net reaching 0.9705 accuracy and 0.9200 macro F1. The post-selection model further refines these results, achieving 0.9720 accuracy and 0.9229 macro F1.This refitting step follows the general post-selection logic: after a penalized variable-selection stage, refitting a model on the selected variables can reduce shrinkage bias and provide a clearer final predictive model (Belloni et Chernozhukov, 2013). Finally, the Random Forest classifier outperforms all linear approaches, reaching 0.9813 accuracy and 0.9486 macro F1. These improvements are not marginal. The increase in macro F1 from 0.8904 to 0.9486 represents a substantial gain in balanced classification performance across all classes. This confirms that the successive methodological steps—regularization, feature selection, and non-linear modelling—are not only theoretically justified, but also empirically effective. However, predictive performance alone is not sufficient to determine the best model. From the perspective of interpretability, multinomial logistic regression models remain superior. The LASSO and Elastic Net solutions provide sparse and stable

representations of the semantic space, identifying a reduced subset of predictors that can be directly interpreted in terms of semantic dimensions. In contrast, Random Forest offers higher predictive accuracy but at the cost of reduced interpretability, since the decision process is distributed across many trees and interactions. From a statistical learning perspective, the results also confirm the importance of controlling multicollinearity and dimensionality. The strong correlations observed among semantic features (22 pairs with $|r| \geq 0.8$ and 51 predictors with VIF ≥ 5) make unregularized models unstable. Regularization and feature selection mitigate this issue by reducing variance and focusing the model on the most informative predictors. Overall, the comparison suggests that no single model is universally optimal. Instead, the choice depends on the objective of the analysis. If the goal is interpretability and semantic insight, regularized multinomial models are preferable. If the goal is predictive accuracy, non-linear ensemble methods such as Random Forest provide the best results.

3 Results

The empirical results of the project provide a coherent picture of the Binder/Testa semantic feature space as a highly informative, but strongly imbalanced and redundant, representation for lexical-semantic classification.

First, the preprocessing phase confirms that the operational dataset is structurally usable and does not require aggressive cleaning. The starting matrix contains 535 lexical entries, 65 semantic predictors, and the target variable `target_word_class`. No exact duplicate rows are removed, while two duplicated word forms are detected and retained because the statistical unit is the lexical entry rather than the orthographic form alone. Missingness is not spread uniformly across the whole feature space, but is concentrated in a small number of variables: `complexity` has 101 missing values, corresponding to about 18.9% of the dataset, while `practice` and `caused` have 39 missing values each, and `drive` has only one. After the stratified train-test split, the preprocessing pipeline handles missing values through median imputation fitted on the training set only, followed by standardization fitted again only on the training data. This avoids data leakage and produces a clean modelling matrix with 428 training observations and 107 test observations.

The most important structural property of the dataset is class imbalance. The full dataset contains 434 nouns, 62 verbs, and 39 adjectives, corresponding respectively to the semantic classes *Thing*, *Action*, and *Property*. Thus, the majority class alone accounts for approximately 81.1% of the observations. The same proportions are preserved in the split: the test set contains 87 nouns, 12 verbs, and 8 adjectives. This explains why the majority-class baseline reaches approximately 0.813 test accuracy despite having no real discriminative ability. Its balanced accuracy is only 0.333, and its macro-F1 is approximately 0.299, because it completely ignores the two minority classes. For this reason, accuracy is informative only when interpreted together with balanced accuracy, macro-F1, class-wise recall, and confusion matrices.

The unsupervised analysis shows that the semantic feature space contains meaningful internal structure, but this structure does not coincide directly with the three lexical classes. PCA on the 65 standardized semantic features shows that the first component explains about 22.7% of the variance and the second about 13.4%, for a combined explained variance of approximately 36.1%. Ten principal components are needed to explain about 75% of the total variance. This indicates that the Binder semantic ratings are structured, but not reducible to a very small number of dimensions. Clustering confirms the same pattern. K-Means reaches its best silhouette score at $k = 14$, with silhouette 0.3099, while hierarchical clustering also prefers a relatively fine partition, again at $k = 14$, with silhouette 0.2381. However, the agreement with the true lexical labels remains weak: for $k = 3$, K-Means obtains ARI 0.0301 and NMI 0.0720, while hierarchical clustering obtains ARI -0.0667 and NMI 0.0425. Even using the best $k = 14$ partitions, the alignment with the lexical labels remains limited. This means that the semantic space captures finer semantic neighborhoods rather than directly reproducing the broad noun/verb/adjective distinction.

The supervised models on the original 65-feature space show that the semantic ratings are highly predictive of lexical class. The unpenalized multinomial logistic regression reaches 0.9533 test accuracy, 0.9191 balanced accuracy, and 0.8904 macro-F1. This is far above the dummy majority baseline and demonstrates that the semantic attributes contain strong class-discriminative information. However, the model also reaches perfect training performance, which suggests that the dataset is easily separable in-sample and that evaluation on held-out data is essential. The regularized LASSO and Elastic Net models obtain the same test performance as the baseline multinomial model on the original 65-feature space. In this specific setting, therefore, regularization does not produce a clear predictive improvement and does not induce true sparsity: both LASSO and Elastic Net keep all 65 features active. This result is statistically plausible because $p = 65$ is still much smaller than $n_{\text{train}} = 428$, so the problem is not yet genuinely high-dimensional. The main difficulty in the original feature space is not dimensionality, but class imbalance and multicollinearity.

The multicollinearity diagnostics confirm that the original semantic predictors are correlated in interpretable ways. Among the 2080 feature pairs, 199 have absolute correlation at least 0.50, 53 at least 0.70, 22 at least 0.80, and 2 at least 0.90. The maximum absolute correlation is approximately 0.937. VIF diagnostics also show non-negligible redundancy: 51 features have VIF above 5, and 17 above 10, with a maximum finite VIF of about 23.0. This confirms that semantic attributes are not independent measurements. Perceptual, motor, social, affective, and cognitive dimensions tend to co-occur in word meanings, so coefficient-based interpretations must be treated carefully.

The high-dimensional expansion changes the nature of the problem more substantially. By adding squared terms and all pairwise interactions, the feature space increases from 65 to 2210 predictors. With 428 training observations, this creates a true $p > n$ setting. The expanded matrix is necessarily rank-deficient, with a lower-bound rank deficiency of 1782, and classical VIF cannot be computed because $X^\top X$ is singular. Pairwise correlations become much more numerous: 127467 pairs have absolute correlation at least 0.50, 28737 at least 0.70, 9962 at least 0.80, 1945 at least 0.90, and 404 at least 0.95. This confirms that the expansion introduces both useful semantic interactions and substantial redundancy.

Despite this redundancy, the high-dimensional representation is strongly predictive. A Ridge logistic model fitted on all 2210 features reaches 0.9720 test accuracy, 0.9646 balanced accuracy, and 0.9290 macro-F1. Its confusion matrix shows only three errors: two *Things* are classified as *Properties*, and one *Action* is classified as *Property*. This improvement over the 65-feature baseline suggests that nonlinear terms and interactions contain additional class information. In semantic terms, this is plausible because lexical classes are unlikely to depend only on isolated attributes. For example, action-like meanings may be better captured by combinations involving `practice`, `body`, `biomotion`, and `path`, while property-like or abstract meanings may emerge from combinations involving `human`, `self`, `cognition`, `caused`, and `consequential`.

Feature screening is therefore essential in the high-dimensional phase. The screening step reduces the feature space from 2210 predictors to 300, removing 1910 variables while retaining the terms with the strongest marginal evidence. The highest-ranked features are almost entirely interaction terms, including `practice_x_human`, `short_x_human`, `practice_x_caused`, `practice_x_short`, `practice_x_consequential`, `biomotion_x_short`, `short_x_self`, `near_x_short`, `practice_x_self`, and `body_x_short`. This is one of the clearest results of the project: the most informative high-dimensional predictors are not arbitrary polynomial artifacts, but interpretable semantic combinations. The selected terms are strongly connected to action structure, embodiment, agency, causality, spatial-temporal organization, and self/human-related conceptual content.

After screening, LASSO and Elastic Net achieve perfect performance on the held-out test set: 1.000 accuracy, 1.000 balanced accuracy, 1.000 macro-F1, and 1.000 weighted-F1. The final post-selection logistic model derived from the LASSO active set also obtains perfect test performance. This result indicates that, within this split, the screened interaction-based semantic representation separates the three classes completely. At the same time, it must be interpreted cautiously. The test set is small, especially for

the minority classes, with only 12 verbs and 8 adjectives. Therefore, perfect performance should not be read as proof that the problem is universally solved, but as evidence that the high-dimensional semantic expansion creates a very strong separating signal in the available data.

The sparsity analysis shows that the high-dimensional regularized models are selective, but not extremely sparse. LASSO retains 234 active features out of the 300 screened predictors, while Elastic Net retains 253. In coefficient terms, LASSO sets about 47.7% of the class-specific coefficients to zero, whereas Elastic Net sets about 41.8% to zero. This behaviour is coherent with the structure of the problem. Because many semantic interactions are correlated and partially redundant, Elastic Net tends to retain groups of related predictors, while LASSO is more aggressive but still keeps a large active set. The result suggests that lexical-class discrimination is distributed across multiple semantic combinations rather than concentrated in a very small number of isolated variables.

Finally, the alternative classifier comparison confirms that the semantic representation is not only effective for multinomial logistic models. On the selected 65-feature semantic space used in the extra comparison, Random Forest achieves the best test accuracy and macro-F1, with 0.9813 accuracy and 0.9486 macro-F1. The linear SVM obtains slightly lower macro-F1, 0.9415, but the best balanced accuracy, 0.9885, because it correctly classifies all adjectives and all verbs, making errors only on nouns. The RBF SVM also performs strongly, with 0.9720 accuracy and 0.9381 macro-F1. The post-selection logistic model remains competitive, with 0.9439 accuracy and 0.8748 macro-F1, while k-NN reaches 0.9533 accuracy but lower balanced accuracy, 0.8194, because it is less reliable on minority classes. These results indicate that the semantic space supports several modelling strategies, but margin-based and ensemble classifiers exploit it particularly well. Overall, the results show that the Binder semantic representation is informative, structured, redundant, and partly nonlinear. The original 65-feature matrix already supports strong lexical-semantic classification, but its correlated structure limits sparse feature selection. The high-dimensional expansion exposes additional discriminative information through semantic interactions, but also creates severe multicollinearity and $p > n$ instability. The best results are obtained when expansion is combined with screening and regularization. Scientifically, this supports the central hypothesis of the project: lexical classes are not predicted only by isolated semantic dimensions, but by structured configurations of perceptual, motor, causal, social, cognitive, and temporal attributes.

4 Discussion

The findings of this project provide several insights into the relation between semantic representation, lexical classification, and high-dimensional statistical learning. From a linguistic point of view, the first important result is that semantic similarity and lexical-class membership are connected, but they are not equivalent. The unsupervised analysis shows that the Binder/Testa feature space naturally contains meaningful semantic structure: PCA and clustering reveal broad gradients and coherent groups of lexical items, such as concrete entities, food-related words, animal names, affective concepts, and action-like meanings. However, these data-driven structures do not align perfectly with the target classes *Thing*, *Action*, and *Property*. This suggests that lexical classes are not simple geometric clusters in the semantic space. Rather, they should be understood as higher-level linguistic categories that are partially grounded in semantic content, but not reducible to semantic similarity alone. This distinction is important for the interpretation of the whole project. If the unsupervised clusters had corresponded directly to the three lexical classes, the classification problem would have been almost trivial: the semantic space itself would have already separated nouns, verbs, and adjectives without supervision. Instead, the weak alignment between clusters and labels indicates a more complex situation. The semantic representation captures fine-grained conceptual organization, whereas the supervised models learn the specific configurations of semantic features that are relevant for lexical-class discrimination. In linguistic terms, the results support an interface view: morphosyntactic or lexical-semantic classes are influenced by conceptual structure, but they are not identical to conceptual similarity. A second central result concerns the role of

multicollinearity. The strong correlations observed among semantic features should not be interpreted merely as a technical problem. They reflect the nature of the semantic representation itself. Experiential dimensions such as perception, motion, body, affect, cognition, social interaction, causality, and attention are conceptually related and often co-occur in word meanings. For example, action-related words may jointly activate bodily, motor, path-related and causal dimensions; affective words may combine emotional, social and cognitive attributes; concrete entity words may involve visual, spatial, tactile and object-related features. Therefore, redundancy in the predictor matrix is not accidental, but is part of the structure of conceptual representation (Binder et al., 2016); (Cree et McRae, 2003); (McRae et al., 1997). From a statistical point of view, however, this redundancy has direct consequences. Correlated predictors make coefficient estimates less stable and complicate feature selection, because several variables may carry partially overlapping information. This explains why regularization is methodologically appropriate in this setting. LASSO is useful because it introduces sparsity, but in the presence of strongly correlated semantic dimensions it may select one predictor among several related alternatives. Elastic Net is better suited to this type of structure because its mixed L_1/L_2 penalty can preserve groups of correlated predictors more stably (Friedman et al., 2010); (Tibshirani, 1996); (Zou et Hastie, 2005). The results therefore confirm that regularization is not only a way to improve prediction, but also a necessary tool for obtaining a more stable interpretation of a redundant semantic feature space. A third insight concerns the distributed nature of lexical-semantic information. The results do not suggest that lexical classes are identified by a small number of isolated semantic dimensions. On the contrary, the strongest evidence comes from combinations of features. The high-dimensional expansion shows that interaction terms involving dimensions such as `practice`, `body`, `biomotion`, `path`, `human`, `self`, `caused`, and `cognition` are among the most informative predictors. This indicates that the distinction between *Thing*, *Action*, and *Property* depends on structured semantic configurations. For instance, action-like meanings are not captured only by a single motion feature, but by the joint contribution of bodily, motor, temporal, path-related and causal information. Similarly, property-like and abstract meanings may emerge from combinations of cognitive, social, evaluative and relational dimensions. This result has a clear neurolinguistic interpretation, although it must be stated cautiously. The Binder features used here are not direct neural measurements, but they are motivated by neurocognitive theories of semantic representation. The fact that these features support lexical classification suggests that neurocognitively motivated semantic dimensions contain information relevant to linguistic category structure. The results are therefore compatible with grounded and distributed accounts of word meaning, according to which conceptual knowledge is organized through partially sensory, motor, affective, social and cognitive systems rather than through purely amodal symbolic labels (Barsalou, 1999); (Binder et al., 2016); (Meteyard et al., 2012). At the same time, the analysis does not show that the brain performs lexical classification through the same statistical mechanisms used by our models. It shows instead that a brain-inspired semantic representation contains enough structured information to make lexical-class prediction possible. The comparison between linear and non-linear models further clarifies the nature of the predictive signal. Linear and regularized multinomial models are valuable because they provide interpretable coefficients and allow the selected semantic dimensions to be inspected directly. They are therefore appropriate when the goal is to understand which semantic attributes contribute to lexical-class discrimination. Non-linear models, especially Random Forest, achieve stronger predictive performance because they can exploit complex interactions among predictors without requiring those interactions to be explicitly specified in a linear formula (Breiman, 2001). The strong performance of SVM models also suggests that, after feature selection, the classes become close to separable in the selected semantic space under a margin-based criterion (Cortes et Vapnik, 1995). The project therefore reveals a trade-off: interpretable linear models are useful for semantic explanation, while more flexible classifiers are useful for maximizing predictive accuracy. Finally, the high-dimensional extension highlights both the opportunities and the risks of increasing model complexity. Expanding the feature space from 65 to 2210 predictors allows the analysis to represent nonlinear effects and interactions between semantic dimensions. This is useful because the results show that many discriminative patterns are interaction-based. However, the same expansion also creates a $p > n$ problem, severe multicollinearity, rank deficiency, and a higher

risk of overfitting. The success of feature screening demonstrates that dimensionality reduction is essential in this context: screening acts as a bridge between semantic expressiveness and statistical stability by reducing the expanded space to a smaller set of informative candidate predictors (Fan et Lv, 2008); (Fan et al., 2009); (Nandy et al., 2022). For this reason, the high-dimensional results should be interpreted as evidence that interaction-based semantic structure is highly informative, but also as a reminder that such results require careful control through screening, regularization and cautious evaluation (Yu et Kumbier, 2020).

Linguistic and neurolinguistic interpretation. From a linguistic perspective, the results suggest that lexical classes are not arbitrary formal labels, but are systematically related to the semantic structure of words. The fact that the original semantic feature space already supports strong supervised classification indicates that nouns, verbs, and adjectives differ not only in their morphosyntactic behaviour, but also in their experiential semantic profiles. Nouns, corresponding to *Things*, tend to be associated with object-like and entity-like semantic configurations; verbs, corresponding to *Actions*, are more naturally connected to eventive, motor, causal and path-related dimensions; adjectives or property-like items are instead linked to attribute-based, evaluative, scalar or more abstract semantic information. Therefore, the classification task does not merely recover grammatical labels from noise, but exploits meaningful semantic regularities that are linguistically interpretable. At the same time, the unsupervised analysis shows that semantic organization and morphosyntactic classification are not the same phenomenon. PCA and clustering reveal coherent semantic structure, including broad gradients such as concreteness versus abstractness and finer semantic groupings, but these structures do not perfectly align with the three lexical classes. This means that the semantic space is organized primarily around dimensions of conceptual content, while lexical classes are better understood as higher-level linguistic categories that partially depend on, but do not coincide with, semantic similarity. In linguistic terms, the results therefore support an interface view: semantic features contribute to the distinction between nouns, verbs and adjectives, but morphosyntactic class membership cannot be reduced to a simple clustering of meanings. The high-dimensional results strengthen this interpretation. The most informative predictors after feature expansion are mostly interaction terms, such as combinations involving `practice`, `human`, `body`, `biomotion`, `path`, `self`, `caused`, and `cognition`. This suggests that lexical-class information is not encoded by single semantic dimensions in isolation, but by configurations of semantic properties. For instance, action meanings are plausibly characterized by the joint presence of bodily, motor, causal and path-related information, rather than by a single feature such as `motion` alone. Similarly, property-like or more abstract meanings may emerge from combinations of cognitive, social, evaluative and relational dimensions. This supports a compositional and distributed view of lexical semantics, where the relevant signal lies in structured interactions among semantic attributes. From a neurolinguistic perspective, these findings are compatible with brain-based and embodied accounts of conceptual representation. The Binder semantic features are not direct neural measurements in this project, but they are cognitively and neurobiologically motivated dimensions derived from theories of perceptual, motor, affective and cognitive grounding (Barsalou, 1999); (Binder et al., 2016); (Meteyard et al., 2012). The fact that these features predict lexical classes suggests that neurocognitively motivated semantic dimensions contain information relevant not only to conceptual similarity, but also to the organization of lexical categories. In this sense, the results are consistent with the idea that lexical knowledge is distributed across partially modality-specific semantic systems, rather than stored as purely amodal symbolic labels. However, the neurolinguistic interpretation must remain cautious. The present analysis does not demonstrate that the brain explicitly classifies words through the same statistical mechanisms used by our models. It also does not provide direct evidence about neural activation patterns. Rather, it shows that a semantic representation inspired by neurocognitive theories contains enough structured information to distinguish lexical classes. This provides indirect support for the hypothesis that the organization of lexical categories may be grounded in distributed semantic information involving sensory, motor, affective, social, temporal and cognitive dimensions. The distinction between the unsupervised and supervised results is especially important in this respect. If the semantic space had clustered perfectly into nouns, verbs and adjectives, one could argue that lexical categories are simply natural semantic clusters. Instead, the weak alignment

between clusters and labels suggests a more nuanced picture: the semantic system appears to be organized around experiential and conceptual dimensions, while morphosyntactic classes emerge from particular configurations of those dimensions. This is compatible with the idea that grammar and semantics interact, but remain distinct levels of linguistic organization. Overall, the linguistic and neurolinguistic conclusion is that lexical classes are semantically grounded but not semantically reducible. The Binder feature space captures meaningful experiential structure; supervised models show that this structure is predictive of lexical class; and high-dimensional interactions reveal that the relevant information lies in combinations of semantic dimensions. Therefore, the project supports a view of lexical classification as an interface phenomenon between conceptual semantics, morphosyntax and neurocognitively grounded representations of meaning.

5 Conclusion

This project has reformulated the analysis of the Binder semantic dataset within the framework of high-dimensional statistical learning. The research question was whether a cognitively motivated semantic representation, originally composed of 65 experiential attributes, could be used not only to describe word meaning, but also to predict broad lexical-semantic classes in a statistically reliable and interpretable way. More specifically, the project asked whether the classification of lexical items into *Thing*, *Action*, and *Property* could benefit from regularization, feature screening, and high-dimensional expansion, while preserving a meaningful interpretation of the selected semantic predictors. Starting from the Binder/Testa semantic matrix, we developed a structured pipeline combining preprocessing, unsupervised exploration, supervised classification, multicollinearity diagnostics, high-dimensional feature expansion, marginal screening, regularization, and model comparison. The first conclusion is that the semantic feature space is genuinely informative. Even the baseline multinomial logistic regression fitted on the original 65 semantic attributes achieves strong test performance, with high accuracy and macro-F1. This shows that the experiential dimensions proposed in the Binder representation contain enough discriminative information to support lexical-class prediction. Therefore, the semantic ratings are not only theoretically meaningful, but also statistically predictive. The second conclusion is that semantic similarity and lexical classification are related but not equivalent. The unsupervised analysis shows that the feature space contains meaningful internal structure: PCA reveals dominant semantic gradients, and clustering identifies coherent semantic regions. However, these unsupervised structures do not align perfectly with the three target classes. This indicates that the Binder feature space captures fine-grained semantic organization, such as concreteness, sensory-motor information, affective content, and conceptual similarity, rather than directly reproducing the noun/verb/adjective distinction. For this reason, supervised models are necessary when the objective is lexical-class prediction. The third conclusion concerns redundancy and multicollinearity. The original 65 semantic features are not independent: many dimensions are naturally correlated because word meanings involve combinations of perceptual, motor, social, affective, and cognitive components. This redundancy is not a defect of the dataset, but a property of semantic representation. From a modelling perspective, however, it makes coefficient interpretation more delicate and motivates the use of regularized models. Ridge, LASSO, and Elastic Net are therefore not only technical alternatives, but appropriate tools for controlling instability in a correlated semantic space. The high-dimensional expansion provides the most important methodological extension of the project. By adding squared terms and pairwise interactions, the feature space increases from 65 to 2210 predictors, producing a genuine $p > n$ setting. This transformation makes the problem statistically more difficult, because it introduces rank deficiency, strong multicollinearity, and a high risk of overfitting. At the same time, it allows the model to represent semantic configurations that cannot be captured by isolated features alone. The screening results show that many of the most informative predictors are interaction terms, such as combinations involving `practice`, `human`, `body`, `biomotion`, `path`, `self`, `caused`, and `cognition`. This supports the idea that lexical classes are not determined only by single semantic dimensions, but by structured combinations of experiential attributes. Feature screening is therefore a central part of the final pipeline. Reducing

the expanded space from 2210 to 300 predictors makes the high-dimensional problem more manageable while preserving the most informative semantic terms. After screening, LASSO and Elastic Net achieve extremely strong test performance, and the final post-selection model confirms the separability of the classes in the screened interaction-based space. However, this result must be interpreted cautiously. The test set is small, especially for the minority classes, and perfect test performance should not be treated as definitive evidence of universal generalization. Rather, it shows that, within the available split, the high-dimensional semantic representation contains a very strong separating signal. The comparison with alternative classifiers further clarifies the nature of the predictive signal. Random Forest achieves the best overall accuracy and macro-F1, while the linear SVM obtains the best balanced accuracy. This suggests that the selected semantic space is highly predictive across different modelling families. The strong performance of Random Forest indicates that nonlinear interactions among semantic attributes are useful for classification, while the strong performance of the linear SVM suggests that, after feature selection, the classes are also close to linearly separable in the selected representation. The dummy majority classifier, despite its high raw accuracy, performs poorly in terms of balanced accuracy and macro-F1, confirming that class imbalance must always be taken into account. Overall, the answer to the research question is positive. The Binder semantic representation can be successfully reformulated as a supervised statistical learning problem, and high-dimensional statistical learning can improve the analysis by exposing nonlinear and interaction-based semantic structure. The original 65-feature space is already highly informative, but the expanded representation reveals that lexical classification depends on combinations of semantic attributes rather than on isolated dimensions alone. The best results are obtained when high-dimensional expansion is combined with screening and regularization, because this strategy balances expressiveness, stability, and interpretability. The project also highlights an important methodological trade-off. Linear and regularized multinomial models are more interpretable and allow a clearer semantic reading of the selected predictors. Nonlinear models, especially Random Forest, achieve slightly stronger predictive performance but are less transparent. Therefore, the best model depends on the objective of the analysis: if the goal is semantic interpretation, regularized and post-selection logistic models are preferable; if the goal is pure predictive accuracy, ensemble and margin-based classifiers are more effective. Future work could extend this analysis in several directions. A larger and more balanced dataset would allow more reliable evaluation of minority classes and more stable validation of the high-dimensional results. Repeated resampling, nested cross-validation, or stability selection could be used to assess how robust the selected semantic interactions are across different splits. Finally, the Binder experiential ratings could be compared with distributional or contextual word representations, in order to evaluate whether cognitively motivated semantic features and corpus-based embeddings capture complementary aspects of lexical meaning. In conclusion, the Binder semantic representation proves to be a rich and informative basis for statistical learning. It supports accurate lexical-class prediction, reveals interpretable semantic structure, and becomes especially powerful when expanded into a controlled high-dimensional feature space. The main result of the project is that lexical-semantic classes are best understood not as the effect of isolated semantic variables, but as the outcome of structured configurations of perceptual, motor, causal, social, cognitive, temporal, and affective dimensions.

References

Barsalou, L. W. (1999). "Perceptual symbol systems". In: *Behavioral and Brain Sciences* 22.4, pp. 577–660.

Belloni, A. and Chernozhukov, V. (2013). "Least squares after model selection in high-dimensional sparse models". In: *Bernoulli* 19.2, pp. 521–547.

Binder, J. R., Conant, L. L., Humphries, C. J., Fernandino, L., Simons, S. B., Aguilar, M., and Desai, R. H. (2016). "Toward a Brain-Based Componential Semantic Representation". In: *Cognitive Neuropsychology* 33.3–4, pp. 130–174. DOI: 10.1080/02643294.2016.1147426.

Breiman, L. (2001). "Random forests". In: *Machine Learning* 45, pp. 5–32.

Cortes, C. and Vapnik, V. (1995). "Support-vector networks". In: *Machine Learning* 20, pp. 273–297.

Cover, T. M. and Hart, P. E. (1967). "Nearest neighbor pattern classification". In: *IEEE Transactions on Information Theory* 13.1, pp. 21–27.

Cree, G. S. and McRae, K. (2003). "Analyzing the factors underlying the structure and computation of the meaning of chipmunk, cherry, chisel, cheese, and cello and many other such concrete nouns". In: *Journal of Experimental Psychology: General* 132.2, pp. 163–201.

Csardi, G. (2023). *isa2: The Iterative Signature Algorithm*. URL: https://cran.r-project.org/web/packages/isa2/index.html.

Fan, J. and Lv, J. (2008). "Sure independence screening for ultrahigh dimensional feature space". In: *Journal of the Royal Statistical Society: Series B* 70.5, pp. 849–911.

Fan, J., Samworth, R., and Wu, Y. (2009). "Ultrahigh dimensional feature selection: Beyond the linear model". In: *Journal of Machine Learning Research* 10, pp. 2013–2038.

Friedman, J., Hastie, T., and Tibshirani, R. (2010). "Regularization paths for generalized linear models via coordinate descent". In: *Journal of Statistical Software* 33.1, pp. 1–22.

Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning*. 2nd ed. Springer.

He, H. and Garcia, E. A. (2009). "Learning from imbalanced data". In: *IEEE Transactions on Knowledge and Data Engineering* 21.9, pp. 1263–1284.

James, G., Witten, D., Hastie, T., and Tibshirani, R. (2013). *An Introduction to Statistical Learning: with Applications in R*. Vol. 103. Springer Texts in Statistics. New York: Springer. ISBN: 978-1-4614-7137-0. DOI: 10.1007/978-1-4614-7138-7.

McRae, K., De Sa, V. R., and Seidenberg, M. S. (1997). "On the nature and scope of featural representations of word meaning". In: *Journal of Experimental Psychology: General* 126.2, pp. 99–130.

Meteyard, L., Cuadrado, S. R., Bahrami, B., and Vigliocco, G. (2012). "Coming of age: A review of embodiment and the neuroscience of semantics". In: *Cortex* 48.7, pp. 788–804.

Nandy, D., Chiaromonte, F., and Li, R. (2022). "Covariate information number for feature screening in ultrahigh-dimensional supervised problems". In: *Journal of the American Statistical Association* 117.539, pp. 1516–1529.

Testa, D. (2026). *Davide Testa LinkedIn Profile*. Public LinkedIn profile, accessed in April 2026. URL: https://it.linkedin.com/in/davide-testa-70293b175.

Tibshirani, R. (1996). "Regression shrinkage and selection via the lasso". In: *Journal of the Royal Statistical Society: Series B* 58.1, pp. 267–288.

Yu, B. and Kumbier, K. (2020). "Veridical data science". In: *Proceedings of the National Academy of Sciences* 117.8, pp. 3920–3929.

Zou, H. and Hastie, T. (2005). "Regularization and variable selection via the elastic net". In: *Journal of the Royal Statistical Society: Series B* 67.2, pp. 301–320.

A Original clustering and biclustering analyses

Testa's original project included an extensive exploratory unsupervised analysis of the Binder semantic feature space. The aim of this part of the work was not to predict lexical classes directly, but to investigate whether the 65 primitive semantic attributes were able to produce interpretable similarity structures among words. In other terms, Testa used unsupervised learning to address the question of whether lexical items that are semantically close in an intuitive sense also appear close when represented through Binder et al.'s componential semantic ratings. The first method applied was k-means clustering. This algorithm partitions the observations into k groups by minimizing the within-cluster sum of squared distances, so that words assigned to the same cluster are expected to have similar semantic profiles. The method was used as an initial exploratory baseline because it is simple, widely used, and provides a direct way to test whether the semantic feature space naturally separates into distinct groups. Before fixing the number of clusters, Testa evaluated different criteria, including within-cluster dissimilarity, the Hartigan index, and the average silhouette score. These diagnostics initially suggested $k = 2$ as a possible solution, but the corresponding partition was too coarse and did not lead to a clear semantic interpretation. A second configuration with $k = 10$ was therefore examined. Although this larger number of clusters began to reveal some semantic organization, the resulting groups remained relatively noisy and difficult to interpret. This indicated that the original 65-dimensional space contained meaningful information, but that k-means was not sufficiently flexible to recover stable semantic categories from it. The limitations of k-means motivated the use of agglomerative hierarchical clustering. Hierarchical clustering was chosen because, unlike k-means, it does not require a single fixed partition to be imposed from the beginning. Instead, it builds a dendrogram representing nested similarity relations among observations, which can then be cut at different levels of granularity. In Testa's original project, an agglomerative approach with complete linkage was used. Complete linkage defines the distance between two clusters according to the maximum distance between their members, and therefore tends to produce relatively compact clusters. As in the k-means analysis, the dendrogram was cut at $k = 2$ and $k = 10$. The two-cluster solution again proved too generic, whereas the ten-cluster solution produced more interpretable semantic groupings. In particular, some clusters could be associated with broad semantic areas such as food, animals, natural events, musical instruments, and sentiment-related words. This result suggested that the semantic ratings did encode meaningful similarity relations, but also that the structure of the original feature space was not sharply separated into clean clusters. A further step was the application of biclustering. This method was introduced because standard clustering groups either observations or variables, whereas biclustering simultaneously clusters rows and columns of a data matrix. This is especially relevant for semantic feature data: two words may be similar only with respect to a specific subset of semantic dimensions, rather than across all 65 features. For example, a group of words may be close in auditory features but not in visual or social features. Biclustering is therefore better suited to identifying local semantic patterns, where subsets of lexical items are associated with subsets of experiential attributes. In Testa's original project, biclustering was performed using the `isa2` package in R, based on the Iterative Signature Algorithm. Among the biclustering approaches tested, this method produced the most coherent semantic output. The biclustering analysis identified a large number of local structures. Testa reported 199 biclusters, many of which were semantically interpretable both in terms of the words grouped together and in terms of the features involved. The eight most relevant or coherent biclusters were manually interpreted as corresponding to semantic domains such as *Time*, *Food and Drinks*, *Animals*, *Sounds*, *Locations*, *Negative Events and Feelings*, *Human Beings* and *Practical Tools*. This result was important because it showed that the Binder feature matrix contains local semantic regularities: groups of words are not only similar globally, but can share specific subsets of experiential dimensions. At the same time, the analysis also revealed substantial overlap among biclusters, meaning that the same lexical item could belong to multiple semantic patterns. This overlap is not necessarily a weakness; rather, it reflects the fact that word meaning is multidimensional and that a single concept can participate in different semantic domains. After these first clustering attempts, Testa introduced Principal Component Analysis as a denoising and dimensionality reduction step. The rationale was that the original semantic space is high enough in dimension to contain redundancy, correlated predictors, and noise. PCA transforms the

original variables into orthogonal linear combinations, ordered by the amount of variance they explain. By projecting the data onto a smaller number of principal components, it becomes possible to retain the dominant structure of the dataset while reducing the influence of less informative variation. In Testa's analysis, the first ten principal components were reported to explain approximately 80% of the total variance, with the first few components carrying the largest share of information. The PCA loadings were then inspected to understand the semantic meaning of the main components. In particular, the first two principal components were visualized through a correlation circle and feature contribution plots. Testa interpreted these two components as reflecting two broad and opposite latent semantic dimensions. One component was mainly associated with temporal, causal, social, emotional and attentional features, and was interpreted as closer to an abstract semantic dimension. The other component was more strongly associated with sensory-motor and spatial features, and was interpreted as closer to a concrete semantic dimension. This observation led to one of the main qualitative conclusions of the original project: the opposition between *Concreteness* and *Abstractness* may represent an important latent structure in the semantic organization of the dataset. PCA was also used as a preprocessing step before repeating the clustering analysis. The reason for doing this was that clustering in the original feature space may be affected by redundancy and multicollinearity, while clustering in a lower-dimensional PCA space may reveal cleaner groupings. Testa therefore applied k-means and hierarchical clustering to the PCA-reduced representation. Hierarchical clustering again produced the most meaningful results. With $k = 2$, the clusters approximately separated abstract from concrete lexical items. With larger values of k, especially between $k = 10$ and $k = 28$, the clusters became more semantically specific and closer to the semantic macro-categories identified in Binder et al.'s dataset. However, this increase in semantic detail came with a reduction in the average silhouette score. Testa therefore interpreted the range between 10 and 28 clusters as a trade-off between quantitative cluster compactness and qualitative semantic interpretability. Overall, the unsupervised analyses in Testa's original project had three main purposes. First, they served as a validation of the Binder semantic feature representation, showing that the 65 experiential attributes were not arbitrary but contained recoverable similarity structures. Second, they provided exploratory evidence that some semantic domains could be identified from the data without using lexical-class labels. Third, they motivated the later supervised analysis by showing that the semantic feature space already contained structure relevant to lexical organization. Among the methods tested, simple k-means was the least successful, hierarchical clustering was more interpretable, PCA improved the clarity of the cluster structure, and biclustering provided the richest local semantic patterns. In the present work, these analyses are reported only as background and exploratory evidence. They are not part of the main methodological contribution, because the focus is shifted from unsupervised semantic grouping to supervised high-dimensional statistical learning. Nevertheless, they remain relevant because they justify the use of the Binder feature space as a meaningful input representation. Testa's original project showed that the semantic ratings contain interpretable structure; the present project starts from that result and asks a different question, namely how regularization, feature screening, rebalancing and stability analysis behave when this semantic representation is expanded into a large-feature-space classification problem.

B Detailed explanation of the Python preprocessing pipeline

This appendix describes in detail the three Python scripts used to implement the preprocessing phase of the project. The scripts correspond to the first three operational steps of the analysis pipeline and are stored as separate modules in order to keep the workflow transparent, reproducible, and easy to inspect. The three scripts are:

— `dataset_initial_cleaning.py`, corresponding to Phase 1.1;
— `dataset_train_test_split.py`, corresponding to Phase 1.2;
— `dataset_imputation_scaling.py`, corresponding to Phase 1.3.

The overall objective of these scripts is not to fit predictive models, but to construct a reliable analytical dataset. For this reason, the preprocessing pipeline is deliberately separated from the modelling pipeline.

In particular, the scripts avoid performing operations that could introduce data leakage, such as imputing missing values or estimating scaling parameters before the train–test split. Each phase saves its own outputs inside the `outputs/I/` directory, so that the intermediate results can be examined and reused by later phases of the project.

B.1 Phase 1.1: Initial dataset cleaning

The first script, `dataset_initial_cleaning.py`, performs the initial structural inspection and cleaning of the 65-feature semantic dataset. The input file is:

`dataset/slld_binder_Xy_65_features.csv`

The script saves its outputs in:

`outputs/I/phase1_1_initial_cleaning/`

This phase is intentionally conservative. Its purpose is not to transform the data statistically, but to verify that the dataset has the expected structure and to produce a clean raw version of the data that can be safely used in later phases.

B.1.1 Path definition and output organization

At the beginning of the script, the input and output paths are defined using the `Path` class from Python's `pathlib` module. This makes the code more readable and more robust than relying on raw string paths.

The script defines the main dataset directory:

```
DATASET_DIR = Path("./dataset")
```

and the output directory:

```
OUTPUT_DIR = Path("./outputs/I//phase1_1_initial_cleaning")
```

The double slash in `I//phase1_1_initial_cleaning` does not change the semantic meaning of the path: operating systems normalize it automatically. The important point is that all outputs from Phase 1.1 are stored inside the folder `outputs/I/`, which identifies the first main block of the project pipeline.

The script produces five files:

— `slld_phase1_1_clean_raw.csv`, the structurally cleaned raw dataset;
— `missing_values_report.csv`, a report on missing values by feature;
— `duplicated_word_forms.csv`, a report on repeated word forms;
— `target_class_distribution.csv`, a report on class frequencies;
— `phase1_1_summary.json`, a machine-readable summary of the phase.

The presence of both CSV reports and a JSON summary is useful because the CSV files are easy to inspect manually, while the JSON file can be automatically read by later scripts or used for reproducibility checks.

B.1.2 Expected columns and target validation

The script expects three non-feature columns:

`entry_id`, `word`, `target_word_class`

These columns are stored in the list:

```
ID_COLS = ["entry_id", "word", "target_word_class"]
```

All remaining columns are treated as semantic feature columns. Since the dataset is expected to contain 65 semantic predictors, the script checks whether exactly 65 feature columns are detected. If the number is different from 65, the script does not immediately stop, but prints a warning. This is a sensible choice because a mismatch in the number of predictors may indicate that the wrong input file has been loaded, but it does not necessarily mean that the file is unreadable.

The script also defines the valid target classes as:

```
{"noun", "verb", "adjective"}
```

This check is essential because the whole supervised part of the project assumes a three-class classification task. Rows whose target label is not included in this set are treated as structurally invalid. If such rows are found, the script prints them and removes them. In the prepared dataset, this step is expected to remove no observations.

B.1.3 Loading the dataset and checking file existence

Before loading the dataset, the script checks whether the input file exists. If the file is missing, it raises a `FileNotFoundError`. This is preferable to letting `pandas` fail later with a less informative error message.

The dataset is then loaded using:

```
pd.read_csv(INPUT_FILE)
```

After loading, the script prints the number of rows and columns. This immediate diagnostic is useful because it allows us to verify that the expected dataset has been loaded before any cleaning operation is applied.

B.1.4 Textual cleaning of word and target columns

The script performs a minimal cleaning of the textual columns `word` and `target_word_class`. Specifically, it converts the values to strings and removes accidental leading or trailing whitespace:

```
astype(str).str.strip()
```

This operation is deliberately limited. The script does not lowercase the words, does not lemmatize them, and does not modify their lexical content. This is important because the lexical item is part of the dataset identity. The preprocessing phase should not alter linguistic information unless there is a clear methodological justification.

B.1.5 Exact duplicate rows

The script then checks for exact duplicate rows using:

```
df.duplicated().sum()
```

Exact duplicates are rows that are identical across all columns. If such rows are present, they are removed with:

```
df.drop_duplicates()
```

This choice is methodologically safe because exact duplicates do not introduce new information. Keeping them would artificially increase the weight of repeated observations in later analyses.

It is important to distinguish exact duplicate rows from duplicated word forms. Exact duplicate rows are redundant observations; duplicated word forms may instead correspond to legitimate lexical entries.

B.1.6 Duplicated word forms

After removing exact duplicates, the script searches for duplicated values in the `word` column:

```
df_clean.duplicated("word", keep=False)
```

The result is sorted by word and target class and saved to:

```
duplicated_word_forms.csv
```

Duplicated word forms are reported but not removed. This is a crucial methodological decision. The statistical unit of this project is not simply the written word form, but the lexical entry. Therefore, the same orthographic word can be validly represented more than once if it corresponds to different lexical-semantic roles. For instance, a word may occur once as a verb and once as an adjective. Removing one of these entries would incorrectly collapse distinct lexical uses.

B.1.7 Missing-value analysis

The script computes missing values only on the feature columns, not on the identifier columns. For each semantic feature, it calculates:

— the number of missing values;
— the percentage of missing values.

The missing-value report is sorted in descending order, so that the most problematic features appear first. The report is saved as:

```
missing_values_report.csv
```

The script also computes:

— the number of rows with at least one missing value;

— the number of complete rows.

This information is important because it quantifies the cost of complete-case analysis. If rows with missing values were removed immediately, the sample size would decrease. Since the dataset is small and class-imbalanced, aggressive deletion would be statistically undesirable.

For this reason, Phase 1.1 does not impute missing values and does not remove incomplete rows. Missing values are only described and preserved for later treatment.

B.1.8 Class distribution

The script computes the distribution of `target_word_class` using:

`value_counts()`

It then converts the counts into percentages and saves the output to:

`target_class_distribution.csv`

This step is essential because the target distribution determines how model performance must be interpreted. If one class is much more frequent than the others, raw accuracy can be misleading. A classifier can obtain high accuracy simply by predicting the majority class. Therefore, the class distribution report produced in Phase 1.1 motivates the use of balanced accuracy, macro-F1, class-wise recall, and confusion matrices in later phases.

B.1.9 Saving the clean raw dataset

At the end of the script, the cleaned dataset is saved as:

`slld_phase1_1_clean_raw.csv`

This file is called *clean raw* because it is structurally cleaned but not statistically transformed. In particular, the script explicitly avoids:

— removing rows with missing values;
— removing columns with missing values;
— imputing missing values;
— standardizing features;
— performing a train–test split.

This design is methodologically correct because imputation and scaling must be performed after the train–test split, and they must be fitted only on the training set.

B.1.10 Summary JSON

Finally, the script saves a JSON file containing the main numerical information and methodological decisions of Phase 1.1. The JSON summary includes:

— input and output file paths;
— initial and cleaned dataset dimensions;
— number of identifier columns and feature columns;

— valid target classes;
— number of exact duplicates removed;
— number of duplicated word-form rows;
— number of rows with missing values;
— list of columns with missing values;
— explicit methodological decisions.

This JSON file makes the preprocessing phase auditable. Instead of relying only on the printed console output, the project stores a structured summary that can be inspected later.

B.1.11 Pseudocode for Phase 1.1

Algorithm 1 Phase 1.1: Initial dataset cleaning

```
Define input and output paths
Create output directory if it does not exist
Load the 65-feature dataset
Check that required columns are present
Identify feature columns as all non-identifier columns
if number of feature columns is not 65 then
    Print warning
end if
Strip accidental whitespace from word and target_word_class
Check that all target labels are valid
if invalid target labels are found then
    Remove invalid rows
end if
Count exact duplicate rows
Remove exact duplicate rows
Identify duplicated word forms
Save duplicated word-form report
Compute missing-value counts and percentages by feature
Save missing-value report
Compute target class distribution
Save target class distribution report
Save structurally cleaned raw dataset
Save JSON summary of the phase
```

B.2 Phase 1.2: Stratified train–test split

The second script, `dataset_train_test_split.py`, loads the structurally cleaned raw dataset produced in Phase 1.1 and creates a reproducible train–test split. Its input file is:

`outputs/I/phase1_1_initial_cleaning/slld_phase1_1_clean_raw.csv`

The outputs are saved in:

`outputs/I/phase1_2_train_test_split/`

This phase is crucial because all later preprocessing operations must be fitted only on the training set. Therefore, the split is performed before imputation and before standardization.

B.2.1 Purpose of the split

The script separates the data into:

— a training set, used to estimate preprocessing parameters and fit models;
— a test set, used only for external evaluation.

The split is stratified by the target variable `target_word_class`. Stratification is necessary because the dataset is strongly class-imbalanced. Without stratification, the random split could allocate too few verbs or adjectives to the test set, making evaluation unstable and potentially misleading.

B.2.2 Configuration

The script defines the following main configuration parameters:

— `TEST_SIZE = 0.20`, meaning that 20% of the observations are assigned to the test set;
— `RANDOM_STATE = 42`, ensuring reproducibility;
— `TARGET_COL = "target_word_class"`, defining the stratification variable.

The use of a fixed random state is important because it guarantees that the same train–test split can be reproduced in future runs. Reproducibility is particularly important in a small and imbalanced dataset, where different splits could lead to noticeably different model performance.

B.2.3 Helper function for class distribution

The function:

```
compute_class_distribution(df, split_name)
```

computes, for a given dataset split:

— the count of each class;
— the percentage of each class;
— the name of the split to which the report refers.

The function returns a `pandas` DataFrame. This design avoids repeated code and ensures that the class distributions for the full dataset, training set, and test set are computed in the same way.

B.2.4 Helper function for missing values by split

The function:

```
compute_missing_summary(df, feature_cols, split_name)
```

computes missing-value counts and percentages for each feature within a given split. This is important because missingness may not be distributed identically between the training and test sets. By saving missing-value reports after the split, the project can verify how many missing values will have to be handled in each subset.

B.2.5 Structural checks

After loading the cleaned raw dataset, the script repeats some structural checks:

— it verifies that the identifier columns are present;
— it checks that all target labels are valid;
— it identifies the feature columns;
— it checks whether 65 feature columns are detected.

Repeating these checks is not redundant. Since Phase 1.2 depends on the output of Phase 1.1, these checks ensure that the correct file has been loaded and that the file has not been modified accidentally.

B.2.6 Minimum class-count check

Before performing the stratified split, the script calculates the minimum class count:

```
df[TARGET_COL].value_counts().min()
```

If any class has fewer than two observations, a stratified train–test split would not be feasible. The script therefore raises an error in this case. This is a defensive programming choice: it prevents the procedure from silently producing an invalid split.

B.2.7 Stratified train–test split

The split is performed using `train_test_split` from `sklearn.model_selection`. The key argument is:

```
stratify=df[TARGET_COL]
```

This ensures that the relative class frequencies in the training and test sets remain close to those in the full dataset.

The script uses:

```
shuffle=True
```

so that observations are randomly assigned to train and test while preserving class proportions.

After splitting, both datasets are sorted by `entry_id`. This sorting does not affect the statistical content of the split, but improves readability and makes saved files easier to inspect.

B.2.8 Saving raw train and test datasets

The script saves:

— `slld_phase1_2_train_raw.csv`;
— `slld_phase1_2_test_raw.csv`.

These files are still raw with respect to missing values and scaling. They are separated into train and test, but no imputation and no standardization have been performed. This is the correct order of operations because the imputer and scaler will later be fitted only on the training set.

B.2.9 Class distribution after splitting

The script computes class distributions for:

— the full dataset;
— the training set;
— the test set.

These reports are concatenated and saved in:

`split_class_distribution.csv`

This file allows us to verify empirically that stratification worked as intended. The expected outcome is that the percentage of nouns, verbs, and adjectives remains approximately the same across the three splits.

B.2.10 Majority-class baseline

The script also computes the majority-class baseline. It identifies the most frequent class in the full dataset and calculates the accuracy that would be obtained by always predicting that class.

This value is saved in the summary JSON under:

`majority_class_baseline`

The majority baseline is important because it gives a minimal benchmark for later models. In an imbalanced classification task, a model must not be considered successful simply because it achieves high accuracy. It must improve over the majority baseline and, more importantly, it must perform adequately on minority classes.

B.2.11 Missing values by split

The script computes missing-value summaries for the full dataset, training set, and test set. These summaries are concatenated and saved as:

`missing_values_by_split.csv`

This step prepares Phase 1.3. By knowing which features contain missing values in the training and test sets, we can later verify whether imputation has removed all missing entries.

B.2.12 Summary JSON

The Phase 1.2 JSON summary records:

— input and output paths;
— number of rows in the full dataset, training set, and test set;
— number of columns and feature columns;
— test size and random state;
— stratification column;
— majority-class baseline;
— class distributions;

— missing-value counts by split;
— methodological decisions.

The most important methodological decision recorded in this phase is that imputation, standardization, and feature selection are not performed during splitting. They are explicitly postponed to later phases.

B.2.13 Pseudocode for Phase 1.2

Algorithm 2 Phase 1.2: Stratified train–test split

```
Define input and output paths
Create output directory if it does not exist
Load the cleaned raw dataset from Phase 1.1
Check required columns and valid target classes
Identify feature columns
Compute class distribution of full dataset
Check that each class has enough observations for stratification
Split data into train and test using stratification by target class
Sort train and test sets by entry_id
Save raw training dataset
Save raw test dataset
Compute class distributions for full, train, and test sets
Save class-distribution report
Compute majority-class baseline accuracy
Compute missing-value summaries by split
Save missing-value report by split
Save JSON summary of the phase
```

B.3 Phase 1.3: Imputation and standardization

The third script, `dataset_imputation_scaling.py`, performs missing-value imputation and feature standardization. It uses as input the raw training and test files produced by Phase 1.2:

`slld_phase1_2_train_raw.csv`

and

`slld_phase1_2_test_raw.csv`

The outputs are saved in:

`outputs/I/phase1_3_imputation_scaling/`

This script is the first phase in which statistical transformations are fitted. For this reason, it is the most important preprocessing phase from the point of view of leakage control.

B.3.1 Input validation

The script first checks that both train and test input files exist. If either file is missing, it raises a `FileNotFoundError` and instructs the user to run Phase 1.2 first.

After loading the datasets, the script verifies that the required identifier columns are present in both train and test. It then identifies feature columns separately in the two datasets and checks that they match exactly. This is important because train and test must have the same predictors in the same order before imputation and scaling are applied.

B.3.2 Feature-column conversion to numeric

Before imputation, all feature columns are converted to numeric values using:

```
pd.to_numeric(..., errors="coerce")
```

This operation ensures that the semantic predictors are treated as numerical variables. If a non-numeric value is accidentally present in a feature column, it is converted to `NaN`. This is useful because the imputation step can then handle it consistently as a missing value.

The choice `errors="coerce"` is defensive: it prevents the script from failing on isolated malformed numerical entries and instead includes them in the missing-value treatment.

B.3.3 Missing values before imputation

The script defines a helper function:

```
compute_missing_report(df, feature_cols, split_name, stage)
```

This function computes the missing-value count and percentage for each feature. It also records both the split name and the stage of processing, for example:

— `train`, `before_imputation`;
— `test`, `before_imputation`;
— `train`, `after_imputation`;
— `test`, `after_imputation`.

This design makes it possible to combine all missing-value diagnostics in a single file and directly compare the situation before and after imputation.

The script also computes the number of rows with at least one missing value in both train and test. This provides a row-level measure of missingness, complementing the feature-level report.

B.3.4 Median imputation fitted on training data

The script uses:

```
SimpleImputer(strategy="median")
```

from `sklearn.impute`. Median imputation is chosen because it is simple, robust, and appropriate for numerical semantic ratings. Compared with mean imputation, it is less sensitive to skewed distributions or outlying values.

The most important methodological point is that the imputer is fitted only on the training feature matrix:

```
imputer.fit_transform(train_df[feature_cols])
```

The test set is then transformed using the already fitted imputer:

```
imputer.transform(test_df[feature_cols])
```

This distinction between `fit_transform` on train and `transform` on test is essential. The median values used for imputation must be estimated without looking at the test set. If test-set values were used to compute imputation medians, information from the evaluation data would enter the preprocessing pipeline, creating data leakage.

B.3.5 Reconstructing imputed datasets

The imputer returns NumPy arrays. The script converts these arrays back into `pandas` DataFrames using the original feature names and row indices. It then concatenates the identifier columns with the imputed features.

This produces two complete datasets:

— `train_imputed_df`;
— `test_imputed_df`.

Both retain the original columns `entry_id`, `word`, and `target_word_class`, followed by the 65 imputed semantic features.

B.3.6 Saving imputation values

The script saves the imputation statistics in:

```
imputation_values.csv
```

For each feature, this file reports:

— the feature name;
— the imputation strategy;
— the median value fitted on the training set;
— the number of missing values in the training set before imputation;
— the number of missing values in the test set before imputation.

This file is important for reproducibility and interpretability. It allows us to know exactly which value was used to replace missing entries in each feature.

B.3.7 Missing values after imputation

After imputation, the script recomputes missing-value reports for train and test. It concatenates the before-imputation and after-imputation reports and saves them as:

```
missing_values_before_after.csv
```

The expected result is that both train and test contain zero missing values after imputation. This is verified both feature by feature and at the row level.

B.3.8 Saving imputed but unscaled datasets

The script saves the imputed datasets before scaling:

— `slld_phase1_3_train_imputed.csv`;
— `slld_phase1_3_test_imputed.csv`.

These files are useful because not all later methods require standardized predictors. For instance, tree-based models such as Random Forests do not require feature scaling. Keeping the imputed but unscaled version of the data avoids forcing all later analyses to use standardized variables.

B.3.9 Standardization fitted on training data

The second transformation performed by the script is standardization. The script uses:

```
StandardScaler()
```

from `sklearn.preprocessing`. Standardization transforms each feature according to:

$$z_{ij} = \frac{x_{ij} - \hat{\mu}_{j,\text{train}}}{\hat{\sigma}_{j,\text{train}}},$$

where $\hat{\mu}_{j,\text{train}}$ and $\hat{\sigma}_{j,\text{train}}$ are the mean and standard deviation of feature j computed on the training set only.

As with imputation, the scaler is fitted only on the training set:

```
scaler.fit_transform(train_imputed_df[feature_cols])
```

The test set is transformed using the training-set scaling parameters:

```
scaler.transform(test_imputed_df[feature_cols])
```

This is necessary to preserve the test set as external data. The test set must not be centered and scaled using its own mean and standard deviation, because doing so would allow information from the test distribution to influence the preprocessing.

B.3.10 Why standardization is necessary

Standardization is required because many later statistical learning procedures are scale-sensitive. In particular:

— PCA depends on variances and covariances;
— clustering depends on distances between observations;
— k-nearest neighbors depends directly on the metric structure of the feature space;
— Ridge, LASSO, and Elastic Net penalize coefficients, and the penalty is not comparable across variables measured on different scales.

Without standardization, features with larger numerical dispersion could dominate the analysis, not because they are more semantically important, but because they are measured on a larger scale.

B.3.11 Saving scaling parameters

The script saves the mean and standard deviation used for each feature in:

`scaling_parameters.csv`

For each feature, the file reports:

— the feature name;
— the training-set mean used for scaling;
— the training-set standard deviation used for scaling.

This makes the standardization fully reproducible.

B.3.12 Saving standardized datasets

The standardized datasets are saved as:

— `slld_phase1_3_train_scaled.csv`;
— `slld_phase1_3_test_scaled.csv`.

These are the main inputs for PCA, clustering, distance-based models, and regularized regression/classification models.

B.3.13 Diagnostics on standardized training data

After scaling, the script computes diagnostic quantities on the standardized training matrix:

— the maximum absolute feature mean;
— the minimum feature standard deviation;
— the maximum feature standard deviation.

Since the scaler is fitted on the training set, the training features should have means close to zero and standard deviations close to one. Small deviations from exact zero or one are expected only because of floating-point numerical precision.

These diagnostics are saved in the summary JSON and printed to the console.

B.3.14 Summary JSON

The Phase 1.3 JSON summary records:

— input and output paths;
— train and test dimensions;
— number of feature columns;
— imputation strategy;
— confirmation that imputation is fitted only on the training set;
— standardization method;
— confirmation that standardization is fitted only on the training set;
— missing-value counts before and after imputation;
— standardization diagnostics;
— methodological decisions.

The summary explicitly states that PCA, clustering, feature selection, and model training are not performed in this phase. This reinforces the modular structure of the pipeline: Phase 1.3 prepares the data, while later phases analyze and model them.

B.3.15 Pseudocode for Phase 1.3

Algorithm 3 Phase 1.3: Imputation and standardization

```
Define input and output paths
Create output directory if it does not exist
Load raw training and test datasets from Phase 1.2
Check required columns in train and test sets
Identify feature columns in train and test
if train and test feature columns do not match then
    Stop with an error
end if
Convert all feature columns to numeric values
Compute missing-value reports before imputation
Count rows with at least one missing value in train and test
Initialize median imputer
Fit imputer on training features only
Transform training features
Transform test features using training-fitted imputer
Reconstruct imputed train and test datasets
Save imputation values
Compute missing-value reports after imputation
Save before/after missing-value report
Save imputed but unscaled train and test datasets
Initialize standard scaler
Fit scaler on imputed training features only
Transform imputed training features
Transform imputed test features using training-fitted scaler
Reconstruct standardized train and test datasets
Save scaling parameters
Save standardized train and test datasets
Compute standardization diagnostics on training set
Save JSON summary of the phase
```

B.4 Overall logic of the preprocessing pipeline

The three scripts implement a strict separation between structural cleaning, data splitting, and statistical transformation. This order is essential for a valid supervised learning analysis.

In Phase 1.1, we only inspect and clean the dataset structurally. We remove exact duplicates, report duplicated word forms, inspect missingness, and save the class distribution. No train–test split, imputation, or scaling is performed.

In Phase 1.2, we split the structurally cleaned data into training and test sets. The split is stratified because the target classes are imbalanced. Still, no imputation or scaling is performed.

In Phase 1.3, we finally fit the imputer and scaler. Both are fitted exclusively on the training set and then applied to the test set. This guarantees that the test set does not influence preprocessing parameters.

The complete preprocessing pipeline can be summarized as follows:

Algorithm 4 Complete preprocessing workflow

Load original 65-feature semantic dataset
Perform structural cleaning
Save clean raw dataset
Create stratified train–test split
Save raw train and test datasets
Fit imputer on train only
Apply imputer to train and test
Save imputed train and test datasets
Fit scaler on imputed train only
Apply scaler to train and test
Save standardized train and test datasets
Use standardized data for PCA, clustering, regularized models, and distance-based classifiers
Use imputed unscaled data when scaling is not required

This structure ensures that the preprocessing stage is reproducible, transparent, and statistically valid. It also creates multiple versions of the dataset, each suited to different later methods: raw split data for diagnostics, imputed data for models that do not require scaling, and standardized data for PCA, clustering, distance-based learning, and regularized classification.

C Detailed explanation of the Python analysis pipeline

This section continues the documentation of the Python workflow after the preprocessing pipeline. Once the clean, imputed, and standardized datasets have been produced, the project proceeds through unsupervised exploration, supervised classification, multicollinearity diagnostics, feature screening, regularized modelling, sparsity analysis, final model evaluation, alternative classifiers, and supervised classification based on clustering-derived labels.

The scripts documented in this section are:

— `phase2_unsupervised.py`, corresponding to Phase 2;
— `phase3_1_supervised_data_loading_checks.py`, corresponding to Phase 3.1;
— `phase3_2_train_test_baseline.py`, corresponding to Phase 3.2;
— `phase3_3_multicollinearity_analysis.py`, corresponding to Phase 3.3;
— `phase3_4_feature_screening.py`, corresponding to Phase 3.4;
— `phase3_5_regularized_models.py`, corresponding to Phase 3.5;
— `phase3_6_sparsity_analysis.py`, corresponding to Phase 3.6;
— `phase3_7_final_model_evaluation.py`, corresponding to Phase 3.7;
— `phase3_extra_classifiers_comparison.py`, corresponding to the extra classifier comparison;
— `phase4_cluster_label_supervised_analysis.py`, corresponding to Phase 4.

The guiding principle remains the same as in preprocessing: each script reads explicitly defined inputs, creates a dedicated output directory, saves tabular reports and figures, and writes a JSON summary. This makes the pipeline auditable and reproducible.

C.1 Phase 2: Unsupervised exploration of the semantic space

C.1.1 File: `phase2_unsupervised.py`

The script `phase2_unsupervised.py` performs the unsupervised exploration of the semantic space. Its input is the standardized training dataset produced by Phase 1.3:

`outputs/I/phase1_3_imputation_scaling/slld_phase1_3_train_scaled.csv`

The outputs are saved in:

`outputs/II/phase2_unsupervised/`

The goal of this phase is exploratory. The script does not fit a supervised classifier and does not use the target labels for model training. Instead, it investigates whether the standardized semantic feature space contains interpretable structure. In particular, it applies Principal Component Analysis, K-Means clustering, and Agglomerative Hierarchical Clustering. The known word-class labels are used only after clustering, to evaluate how much the unsupervised structure agrees with the original lexical-semantic classes.

C.1.2 Path definition and configuration

The script defines the input directory:

```
PHASE1_3_DIR = Path("./outputs/I/phase1_3_imputation_scaling")
```

and the output directory:

```
OUTPUT_DIR = Path("./outputs/II/phase2_unsupervised")
```

The main configuration parameters are:

— `ID_COLS`, containing `entry_id`, `word`, and `target_word_class`;
— `TARGET_COL = "target_word_class"`;
— `N_COMPONENTS_FULL = 65`, meaning that full PCA is initially computed using all original semantic dimensions;
— `VAR_THRESHOLD = 0.75`, used to determine how many principal components are needed to retain approximately 75% of the variance;
— `K_RANGE = range(2, 16)`, defining the range of cluster numbers tested;
— `K_SEMANTIC = 3`, used to compare clustering results with the three known target classes.

C.1.3 Loading the standardized data

The function `load_data()` loads the standardized training set. It separates the identifier and target columns from the semantic predictors. The output is composed of:

— the full DataFrame;
— the feature matrix X;
— the target vector y, used only for post-hoc comparison.

C.1.4 PCA routine

The function `run_pca()` fits PCA on the full standardized matrix. It computes:

— the explained variance ratio of each component;
— the cumulative explained variance;
— the number of components needed to reach 75% cumulative explained variance;

— the cumulative variance explained by the first two components;
— the full PCA scores;
— the two-dimensional PCA representation.

The routine also saves the scree plot and the cumulative variance plot. These figures are used to understand whether the semantic space has a low-dimensional structure or whether variance is distributed across many directions.

C.1.5 PCA visualisation

The function `plot_pca_biplot()` projects lexical items onto PC1 and PC2 and colours the points according to their known lexical-semantic class. This does not make PCA supervised: the labels are used only for visualization.

The function `plot_pca_loadings()` identifies the features that contribute most strongly to PC1 and PC2. This allows the first principal components to be interpreted semantically.

C.1.6 K-Means clustering

The function `search_k_kmeans()` runs K-Means over a range of possible cluster numbers. For each k, it computes:

— within-cluster inertia;
— silhouette score;
— cluster assignments.

It also computes the Hartigan index, which evaluates the decrease in inertia when moving from k to $k+1$. The script saves the elbow plot, silhouette plot, and Hartigan plot.

C.1.7 Agglomerative Hierarchical Clustering

The function `run_ahc()` performs hierarchical clustering using complete linkage and Euclidean distance. It saves a dendrogram and computes silhouette scores for several values of k. The clustering solutions are then plotted on the PC1–PC2 plane.

C.1.8 Comparison with known labels

The function `compare_with_labels()` compares the cluster assignments with the known lexical-semantic labels using:

— Adjusted Rand Index;
— Normalized Mutual Information;
— contingency tables;
— heatmaps of cluster-label correspondence.

This comparison is essential because it shows whether the unsupervised geometry of the data naturally reproduces the *Thing*, *Action*, and *Property* distinction.

C.1.9 Pseudocode for Phase 2

Algorithm 5 Phase 2: Unsupervised exploration

```
Load standardized training data from Phase 1.3
Separate identifier columns from the semantic feature matrix
Fit full PCA on the 65 standardized semantic features
Compute explained variance and cumulative explained variance
Identify the number of components needed to reach 75% variance
Save scree plot and cumulative variance plot
Project observations onto PC1 and PC2
Save PCA biplot and loading plots
Use the retained principal components as reduced clustering space
for each k in the selected range do
    Fit K-Means
    Compute inertia and silhouette score
end for
Save K-Means elbow and silhouette plots
Perform hierarchical clustering with complete linkage
for each k in the selected range do
    Cut dendrogram into k clusters
    Compute silhouette score
end for
Save AHC dendrogram and cluster plots
Compare k = 3 cluster labels with known labels
Compute ARI and NMI
Save contingency tables and heatmaps
Save JSON summary
```

C.2 Phase 3.1: Supervised data loading and preliminary checks

C.2.1 File: `phase3_1_supervised_data_loading_checks.py`

This script starts the supervised part of the pipeline. Its purpose is to load the standardized train and test datasets produced by Phase 1.3, separate predictors from the target variable, and create the supervised modelling objects used by the later phases.

The script expects as input:

`slld_phase1_3_train_scaled.csv`

and

`slld_phase1_3_test_scaled.csv`

The outputs are saved in:

`outputs/III/phase3_1_supervised_data_loading_checks/`

C.2.2 Purpose of the script

The script produces the core supervised objects:

— X_{train}, the training feature matrix;
— X_{test}, the test feature matrix;
— y_{train}, the training target vector;
— y_{test}, the test target vector;
— the list of feature names;
— the list of class names.

It also produces quality-control reports on dimensions, missing values, duplicates, duplicated word forms, class distribution, and non-numeric features.

C.2.3 Path resolution

The function `resolve_phase1_3_dir()` searches multiple candidate directories for the Phase 1.3 train and test files. This makes the script robust to small differences in folder structure. If the required files are not found, the function raises a `FileNotFoundError`.

C.2.4 Loading data

The function `load_data()` loads the train and test CSV files. These files already contain identifier columns, target labels, and standardized numerical semantic features.

C.2.5 Structural validation

The function `validate_required_columns()` checks whether the required columns `entry_id`, `word`, and `target_word_class` exist in both train and test data.

The function `extract_xy()` defines the predictor columns as all columns except the identifier and target columns. It then checks that all predictors are numeric. If any non-numeric predictor is found, the script saves a report and stops, because the subsequent supervised models require numerical matrices.

The function `check_feature_alignment()` verifies that train and test contain the same feature columns in the same order. This is essential because the model fitted on train data must receive the same feature structure at test time.

C.2.6 Diagnostic reports

The function `build_shape_summary()` creates a summary table with the number of observations, total columns, feature columns, missing cells, exact duplicates, duplicated entry IDs, and duplicated word forms.

The function `build_class_distribution()` computes the target distribution separately for train, test, and total data.

The function `build_missing_values_report()` reports any remaining missing values. Ideally, after Phase 1.3, no missing values should remain.

The function `build_duplicate_reports()` distinguishes exact duplicate rows from duplicated word forms. Exact duplicates are potential data-quality problems, while duplicated word forms may correspond to distinct lexical entries.

C.2.7 Plots

The script creates:

— a class-distribution bar plot;
— a missing-values plot by split;
— a top-missingness plot;
— a duplicate-diagnostics plot.

C.2.8 Pseudocode for Phase 3.1

Algorithm 6 Phase 3.1: Supervised data loading and checks

Locate Phase 1.3 train and test files
Load scaled training and test datasets
Check that identifier and target columns are present
Define predictors as all non-identifier columns
Extract $X_{\text{train}}, y_{\text{train}}, X_{\text{test}}, y_{\text{test}}$
Check that all predictors are numeric
Check that train and test feature columns match
Compute dataset shape summary
Compute class distributions
Compute missing-value report
Detect exact duplicate rows
Detect duplicated word forms
Save X, y, feature names, and class names
Save diagnostic reports and plots
Save JSON summary

C.3 Phase 3.2: Multinomial logistic baseline

C.3.1 File: `phase3_2_train_test_baseline.py`

This script fits the first supervised classifier on the original semantic feature space. The model is a multinomial logistic regression, used as a baseline for all later models.

The input files are the X, y, feature-name, and class-name outputs generated by Phase 3.1. The outputs are saved in:

`outputs/III/phase3_2_train_test_baseline/`

C.3.2 Why a multinomial logistic baseline is used

The response variable has three mutually exclusive classes. Therefore, multinomial logistic regression is the natural extension of binary logistic regression to this setting. It provides a transparent statistical baseline and allows the later regularized models to be compared against an interpretable GLM reference.

C.3.3 Data loading and checks

The function `load_phase3_1_outputs()` loads the train/test matrices and target vectors from Phase 3.1.

The function `check_feature_alignment()` verifies that the column order matches the saved feature-name metadata.

The function `check_shapes_and_classes()` verifies that:

— X and y have the same number of rows;
— train and test contain all expected classes;
— the split is valid for supervised multiclass learning.

The function `check_feature_standardization()` verifies that the feature matrices are standardized as expected.

C.3.4 Baseline model fitting

The function `fit_baseline_model()` fits a multinomial logistic regression model. It first attempts an unpenalized model, which is the closest to a pure GLM baseline. If the installed version of `scikit-learn` does not support the syntax, the script tries compatibility alternatives, including a weakly penalized L2 fallback.

C.3.5 Prediction and evaluation

The function `predict_with_probabilities()` produces:

— predicted labels;
— predicted class probabilities;
— correctness indicators;
— maximum predicted probability.

The function `compute_split_metrics()` computes:

— accuracy;
— balanced accuracy;
— macro precision;
— macro recall;
— macro F1;
— weighted precision;
— weighted recall;
— weighted F1.

The script also saves classification reports and confusion matrices.

C.3.6 Cross-validation

The function `run_cross_validation()` performs stratified cross-validation on the training set. This is not used to replace the held-out test evaluation. Instead, it provides a stability diagnostic for the baseline model.

C.3.7 Coefficient extraction

The function `save_coefficients()` saves the multinomial coefficient matrix and identifies the strongest positive and negative coefficients for each class. This provides an initial interpretation of which semantic features push predictions toward each class.

C.3.8 Pseudocode for Phase 3.2

Algorithm 7 Phase 3.2: Baseline multinomial logistic regression

1: Load $X_{\text{train}}, y_{\text{train}}, X_{\text{test}}, y_{\text{test}}$
2: Validate feature alignment
3: Validate class availability
4: Check feature standardization
5: Fit multinomial logistic regression on training data
6: Predict labels and probabilities on train and test data
7: Compute global metrics for train and test
8: Compute class-wise metrics
9: Compute confusion matrices
10: Run stratified cross-validation on training data
11: Save model coefficients
12: Save predictions, metrics, plots, model object, and JSON summary

C.4 Phase 3.3: Multicollinearity diagnostics

C.4.1 File: `phase3_3_multicollinearity_analysis.py`

This script investigates redundancy and linear dependence among semantic predictors. It is performed before feature screening and regularized modelling, because multicollinearity affects coefficient stability and feature selection.

The outputs are stored in:

`outputs/III/phase3_3_multicollinearity_analysis/`

C.4.2 Correlation diagnostics

The function `compute_correlation_diagnostics()` computes:

— the Pearson correlation matrix;
— the absolute correlation matrix;
— all ranked feature pairs by absolute correlation;
— highly correlated pairs;
— counts of feature pairs above selected thresholds.

The script uses several thresholds, including 0.50, 0.70, 0.80, 0.90, and 0.95. A high absolute correlation indicates that two features contain overlapping information.

C.4.3 VIF and partial R^2

The function `compute_vif_table()` computes the Variance Inflation Factor for each feature. For each feature X_j, the script fits an auxiliary regression:

$$X_j \sim X_{-j},$$

where X_{-j} denotes all other predictors. It then computes:

$$VIF_j = \frac{1}{1 - R_j^2}.$$

A high VIF indicates that a feature can be well explained by the other features, and is therefore involved in multicollinearity.

The script also computes the associated partial R^2, defined as:

$$R^2_{\text{partial},j} = 1 - \frac{1}{VIF_j}.$$

C.4.4 Feature clustering

The function `compute_feature_clusters()` clusters the variables using correlation distance:

$$d(X_j, X_k) = 1 - |\text{cor}(X_j, X_k)|.$$

This means that strongly positively or negatively correlated variables are considered close. Complete linkage is used to identify groups of mutually correlated features.

C.4.5 Redundant-feature candidates

The function `compute_redundant_feature_candidates()` creates a conservative diagnostic list of features that show signs of redundancy. A feature is considered potentially redundant if it belongs to a non-singleton correlation cluster, has high pairwise correlations, or has elevated VIF.

This list is not used to automatically remove variables. It is only a diagnostic input for later interpretation.

C.4.6 Pseudocode for Phase 3.3

Algorithm 8 Phase 3.3: Multicollinearity diagnostics

Load supervised feature matrices from Phase 3.1
Compute Pearson correlation matrix on X_{train}
Compute absolute correlation matrix
Extract all upper-triangular feature pairs
Rank pairs by absolute correlation
Count feature pairs above correlation thresholds
for each feature X_j **do**
    Regress X_j on all other features
    Compute R_j^2
    Compute $VIF_j = 1/(1 - R_j^2)$
    Compute partial R^2
end for
Define feature distance as $1 - |\text{correlation}|$
Run hierarchical clustering of features
Cut dendrogram at selected correlation threshold
Identify redundant-feature candidates
Save tables, figures, dendrogram, and JSON summary

C.5 Phase 3.4: Feature screening

C.5.1 File: `phase3_4_feature_screening.py`

This script ranks predictors according to marginal supervised relevance. Since the current original feature space contains only 65 predictors, the screening step mainly provides ranking and diagnostics. However, the code is designed to remain valid if the feature space is later expanded to hundreds or thousands of predictors.

C.5.2 ANOVA F-score

The function `compute_anova_scores()` computes the ANOVA F-score for each feature. This measures how strongly the feature values differ across the target classes.

C.5.3 Mutual information

The function `compute_mutual_information_scores()` computes mutual information between each feature and the target. Mutual information is less restricted than ANOVA because it can capture more general dependencies.

C.5.4 Univariate logistic score

The function `compute_univariate_logistic_scores()` fits one logistic regression model per feature. Each model uses only a single predictor. The feature is then scored according to its univariate predictive performance.

C.5.5 Combined ranking

The function `combine_rankings()` normalizes the different score types and combines them into one global screening score:

$$0.40 \times \text{ANOVA} + 0.40 \times \text{Mutual Information} + 0.20 \times \text{Univariate Logistic Score}.$$

This combined score is used to rank features. If the number of features is greater than 500, only the top 500 are retained. If the number of features is 65, all features are retained but ordered by importance.

C.5.6 Pseudocode for Phase 3.4

Algorithm 9 Phase 3.4: Feature screening

```
Load supervised train and test matrices
for each feature do
    Compute ANOVA F-score
    Compute mutual information with target
    Fit univariate logistic model
    Compute univariate logistic score
end for
Normalize all score types
Combine scores into one screening score
Rank features by combined score
if number of features is greater than 500 then
    Keep top 500 features
else
    Keep all features in ranked order
end if
Compute overlap among rankings
Save rankings, selected feature names, diagnostic plots, and JSON summary
```

C.6 Phase 3.5: Regularized multinomial models

C.6.1 File: `phase3_5_regularized_models.py`

This script fits the main regularized supervised models: LASSO and Elastic Net multinomial logistic regression. The models are fitted on the screened feature set from Phase 3.4.

C.6.2 LASSO model

The function `fit_lasso()` uses `LogisticRegressionCV` with an L1 penalty. The regularization parameter is selected by stratified cross-validation using macro F1 as the scoring criterion.

LASSO is important because it can set coefficients exactly to zero, thereby selecting features automatically.

C.6.3 Elastic Net model

The function `fit_elastic_net()` uses a mixed L1/L2 penalty. The L1 component induces sparsity, while the L2 component stabilizes the model when features are correlated.

This is particularly important in this dataset because Phase 3.3 showed substantial multicollinearity among semantic features.

C.6.4 Model evaluation

The function `evaluate_model()` computes train and test metrics. The script saves predictions, classification reports, confusion matrices, coefficient tables, selected features, CV scores, fitted model objects, and plots.

C.6.5 Pseudocode for Phase 3.5

Algorithm 10 Phase 3.5: LASSO and Elastic Net

```
Load X_train, y_train, X_test, y_test
Load screened feature list from Phase 3.4
Subset train and test matrices to screened features
Define grid of inverse regularization strengths C
Fit LASSO multinomial logistic regression with cross-validation
Fit Elastic Net multinomial logistic regression with cross-validation
for each fitted model do
    Predict train and test labels
    Compute evaluation metrics
    Save test predictions
    Save classification report
    Save confusion matrix
    Save coefficient table
    Identify selected features
    Save selected feature names
    Save CV scores and CV plots
end for
Save fitted models and JSON summary
```

C.7 Phase 3.6: Sparsity analysis

C.7.1 File: `phase3_6_sparsity_analysis.py`

This script does not fit new models. Instead, it analyses the coefficient tables produced by Phase 3.5 to quantify sparsity.

C.7.2 Sparsity summaries

The function `summarize_sparsity()` counts:

- total coefficients;
- zero coefficients;
- non-zero coefficients;
- zero share;
- active features;
- inactive features.

A coefficient is considered non-zero if its absolute value is greater than a small tolerance.

C.7.3 Class-level sparsity

The function `summarize_by_class()` computes the number of non-zero coefficients separately for each class. This helps identify whether the model uses different semantic dimensions to predict different lexical classes.

C.7.4 Shared and model-specific selected features

The function `shared_specific()` compares features selected by LASSO and Elastic Net. Features can be:

— selected by both models;
— selected only by LASSO;
— selected only by Elastic Net;
— inactive in both.

C.7.5 Pseudocode for Phase 3.6

Algorithm 11 Phase 3.6: Sparsity analysis

```
Load LASSO coefficient table
Load Elastic Net coefficient table
for each coefficient do
    Mark as active if absolute value is above tolerance
end for
for each model do
    Count zero and non-zero coefficients
    Count active and inactive features
    Compute sparsity proportions
end for
for each class do
    Count non-zero coefficients
    Compute class-specific sparsity
end for
Compare LASSO and Elastic Net selected features
Identify shared and model-specific features
Build coefficient matrices
Save tables, heatmaps, sparsity plots, and JSON summary
```

C.8 Phase 3.7: Final model evaluation

C.8.1 File: `phase3_7_final_model_evaluation.py`

This script integrates the previous supervised results and constructs the final post-selection classifier.

C.8.2 Post-selection logic

The script loads the selected features from LASSO and Elastic Net and takes their union. A new multinomial logistic regression model is then fitted on this reduced feature set.

This step separates feature selection from final coefficient estimation. The regularized models identify the relevant variables, while the post-selection model refits the classifier using only those selected variables.

C.8.3 Model comparison

The script compares:

— baseline multinomial logistic regression;
— LASSO;
— Elastic Net;
— post-selection logistic regression.

The function `choose_final_model()` selects the best model according to:

1. test macro F1;
2. test balanced accuracy;
3. test accuracy;
4. smaller number of features.

C.8.4 Pseudocode for Phase 3.7

Algorithm 12 Phase 3.7: Final model evaluation

```
Load supervised train and test matrices
Load fitted LASSO and Elastic Net models
Load selected feature lists from both regularized models
Define final selected feature set as their union
Fit post-selection multinomial logistic regression
Evaluate LASSO, Elastic Net, and post-selection model
Add baseline metrics from Phase 3.2 if available
Compare models on test macro F1, balanced accuracy, accuracy, and feature count
Select final model
Save final predictions, confusion matrices, coefficients, selected features, plots, model object, and JSON summary
```

C.9 Phase 3-extra: Alternative classifiers

C.9.1 File: `phase3_extra_classifiers_comparison.py`

This script tests whether non-linear or non-parametric classifiers improve over the final linear classifier. It uses the same selected feature space as Phase 3.7, so that the comparison focuses on the classifier rather than on preprocessing or feature selection.

C.9.2 Classifiers tested

The script evaluates:

— Dummy majority classifier;
— post-selection logistic regression;
— linear SVM;
— RBF-kernel SVM;
— Random Forest;
— k-nearest neighbors.

C.9.3 Model grid

The function `build_model_grid()` defines each classifier and its hyperparameter grid. The grids are intentionally compact because the dataset is small and the goal is controlled comparison rather than exhaustive tuning.

C.9.4 Evaluation

For each model, the script performs cross-validated hyperparameter tuning when a parameter grid is defined. It then evaluates the best estimator on train and test data.

C.9.5 Pseudocode for Phase 3-extra

Algorithm 13 Phase 3-extra: Alternative classifiers

Load scaled train and test datasets
Load selected feature list from Phase 3.7
if selected feature file is unavailable **then**
    Use all numeric semantic features
end if
Extract X_{train}, X_{test}, y_{train}, y_{test}
Define classifiers and hyperparameter grids
for each classifier **do**
    if hyperparameter grid is not empty **then**
        Run GridSearchCV using macro F1
        Select best estimator
    else
        Fit classifier directly
    end if
    Predict train and test labels
    Compute accuracy, balanced accuracy, macro F1, and weighted F1
    Save classification report
    Save confusion matrix
    Save predictions and fitted model
end for
Save model comparison tables and plots
Select best model by test macro F1
Save JSON summary

C.10 Overall logic of the post-preprocessing pipeline

The complete pipeline after preprocessing follows a progressive statistical logic. First, Phase 2 explores the geometry of the feature space without using the target labels for model fitting. This reveals whether the semantic feature space contains natural clusters or low-dimensional latent structure. Second, Phase 3.1 and Phase 3.2 establish a supervised baseline. The baseline is necessary because all later models must be judged relative to a simple, interpretable classifier. Third, Phase 3.3 diagnoses multicollinearity. This justifies the later use of LASSO and Elastic Net, because correlated semantic features can destabilize ordinary coefficient estimates. Fourth, Phase 3.4 ranks features by marginal supervised relevance. This prepares the feature space for regularized modelling and becomes especially important when the predictor space is expanded. Fifth, Phase 3.5 and Phase 3.6 fit and interpret sparse regularized models. These phases connect prediction with interpretability by identifying which semantic dimensions are actually used by the classifier. Sixth, Phase 3.7 selects the final linear model using a transparent rule based primarily on macro F1 and balanced accuracy. Seventh, Phase 3-extra tests whether more flexible classifiers improve performance, thereby evaluating whether the relationship between semantic features and lexical labels is purely linear or involves non-linear interactions. Finally, Phase 4 repeats the supervised analysis using clustering-derived labels. This provides a direct comparison between externally defined lexical-semantic categories and data-driven semantic groupings.

Algorithm 14 Complete analysis workflow after preprocessing

1: Load standardized train and test datasets
2: Explore semantic structure with PCA and clustering
3: Compare unsupervised clusters with original labels
4: Prepare supervised matrices X and target vectors y
5: Fit multinomial logistic baseline
6: Diagnose multicollinearity using correlations, VIF, and dendrograms
7: Rank predictors with supervised screening
8: Fit LASSO and Elastic Net multinomial models
9: Analyse sparsity and selected features
10: Fit final post-selection logistic model
11: Compare all linear supervised models
12: Fit alternative classifiers on selected features
13: Compare linear and non-linear predictive performance
14: Rebuild cluster-based targets
15: Repeat supervised classification with clustering labels
16: Compare original-label and cluster-label predictability

D Additional tables and figures

This appendix reports supplementary diagnostics and visual outputs produced by the preprocessing, unsupervised, supervised and cluster-label analyses. All figures are included using the original filenames generated by the analysis scripts.

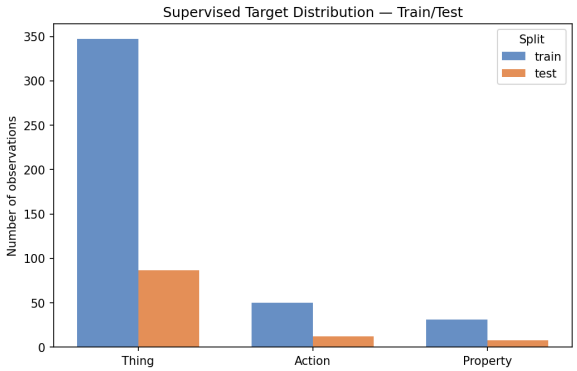
D.1 Preprocessing and supervised data checks

Table 16 – Class distribution after preprocessing and stratified train–test split.

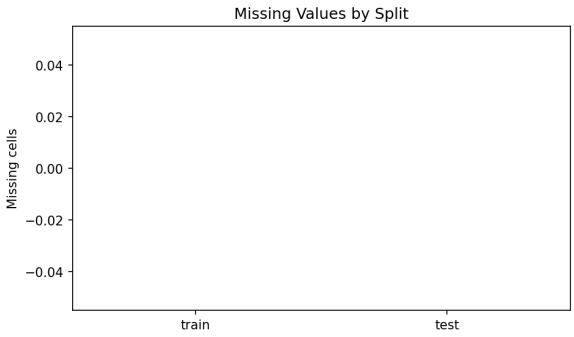
Split	Class	Count	Proportion
Train	Thing	347	0.811
Train	Action	50	0.117
Train	Property	31	0.072
Test	Thing	87	0.813
Test	Action	12	0.112
Test	Property	8	0.075
Total	Thing	434	0.811
Total	Action	62	0.116
Total	Property	39	0.073

Table 17 – Dataset structure after loading the supervised modelling matrices.

Split	Observations	Total columns	ID columns	Features	Missing cells
Train	428	68	3	65	0
Test	107	68	3	65	0
Total	535	68	3	65	0



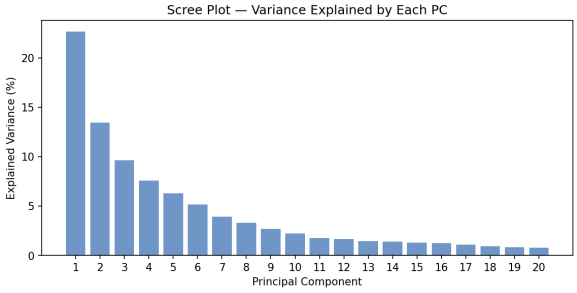
(a) Class distribution.



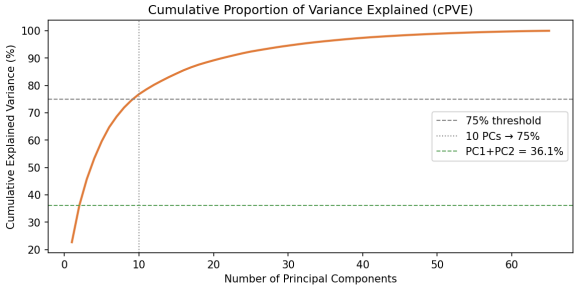
(b) Missing values by split.

Figure 30 – Train–test structure and missing-value diagnostics for the supervised dataset.

D.2 PCA and unsupervised structure



(a) PCA scree plot.



(b) Cumulative explained variance.

Figure 31 – PCA diagnostics used to assess the low-dimensional structure of the semantic feature space.

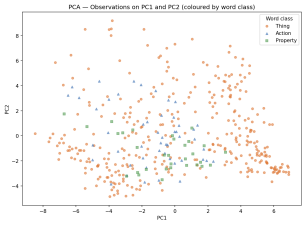
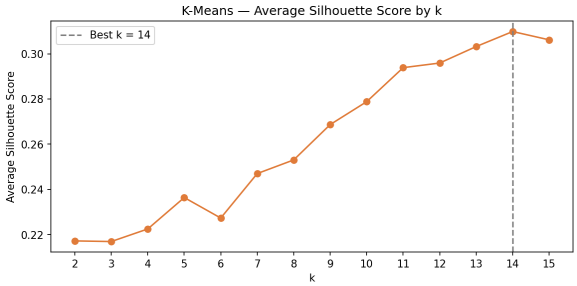
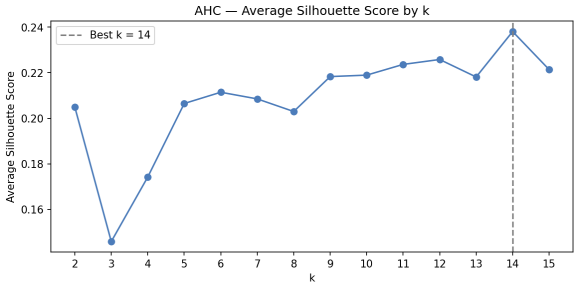


Figure 32 – PCA biplot on the first two principal components.

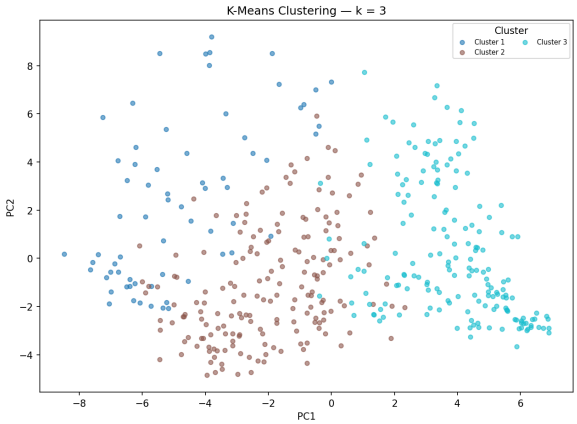


(a) K-Means silhouette profile.

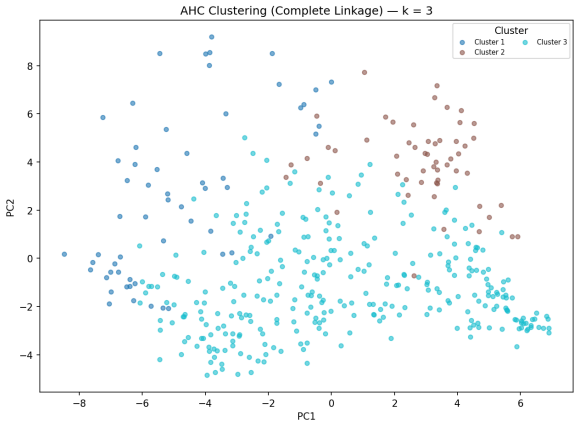


(b) AHC silhouette profile.

Figure 33 – Silhouette-based diagnostics for the unsupervised clustering analyses.

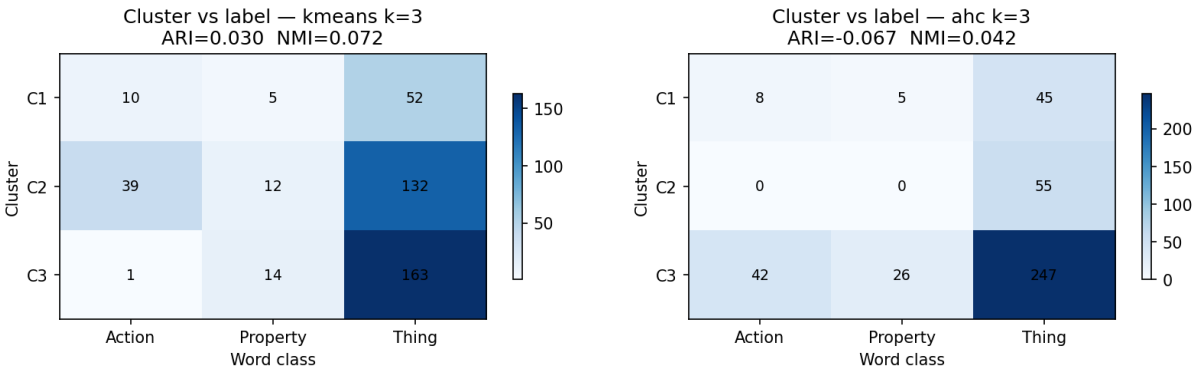


(a) K-Means, $k = 3$.



(b) AHC, $k = 3$.

Figure 34 – Cluster assignments projected on the first two principal components.



(a) K-Means clusters versus lexical labels.

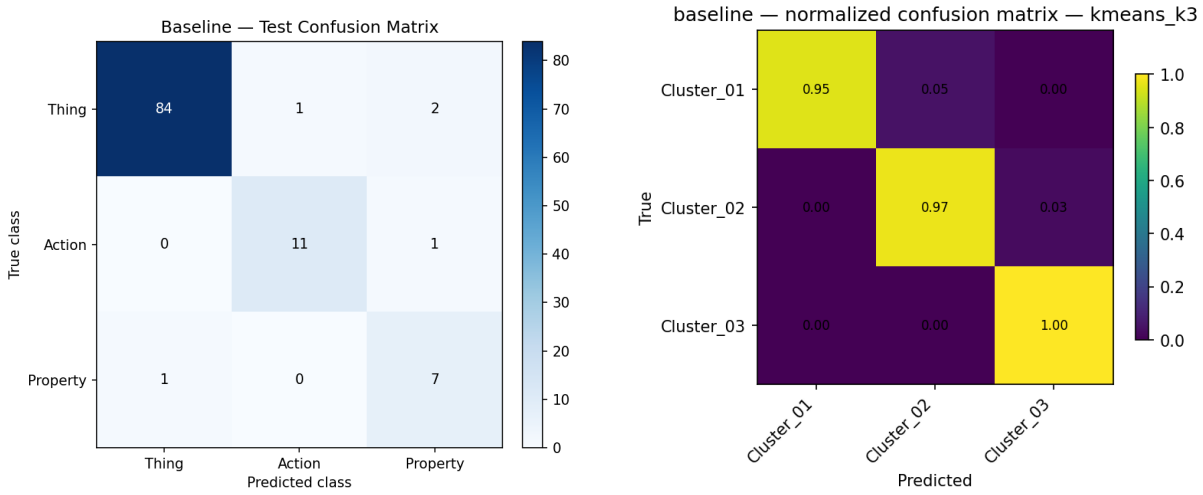
(b) AHC clusters versus lexical labels.

Figure 35 – Association between unsupervised clusters and the original lexical-semantic classes.

D.3 Baseline supervised model

Table 18 – Multinomial baseline performance on train and test sets.

Split	Accuracy	Balanced accuracy	Macro precision	Macro recall	Macro F1
Train	1.000	1.000	1.000	1.000	1.000
Test	0.953	0.919	0.868	0.919	0.890



(a) Absolute confusion matrix.

(b) Normalized confusion matrix.

Figure 36 – Test-set confusion matrices for the baseline multinomial model.

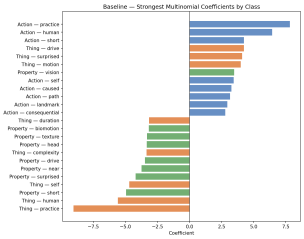


Figure 37 – Largest baseline coefficients by lexical-semantic class.

D.4 Multicollinearity diagnostics

Table 19 – Counts of feature pairs exceeding absolute-correlation thresholds.

Absolute-correlation threshold	Number of pairs	Share of all pairs
0.50	199	0.0957
0.70	53	0.0255
0.80	22	0.0106
0.90	2	0.0010
0.95	0	0.0000

Table 20 – Top ten absolute Pearson correlations among semantic features.

Feature 1	Feature 2	Absolute correlation
pleasant	happy	0.937
face	speech	0.921
audition	sound	0.890
loud	sound	0.888
audition	loud	0.879
motion	fast	0.871
biomotion	body	0.869
unpleasant	disgusted	0.861
unpleasant	angry	0.854
pattern	texture	0.850

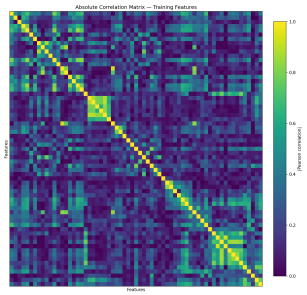
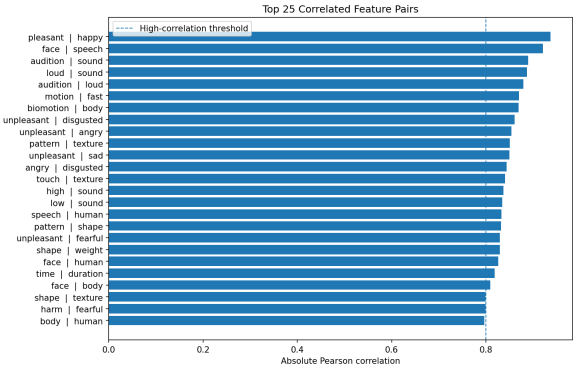
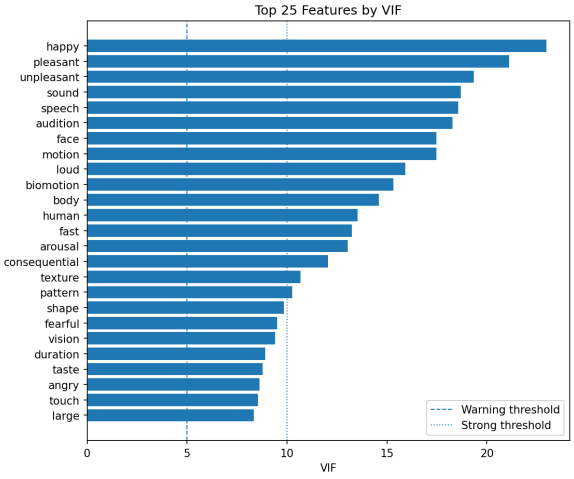


Figure 38 – Absolute correlation structure among the 65 semantic features in the training set.



(a) Top absolute correlations.



(b) Top VIF values.

Figure 39 – Main pairwise and multivariate collinearity diagnostics.

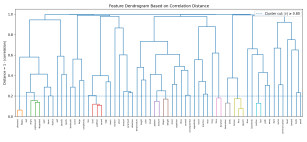
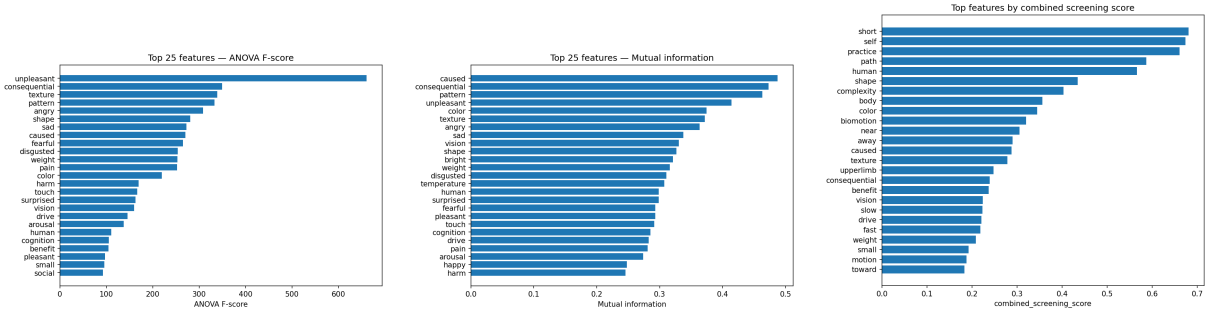


Figure 40 – Hierarchical clustering of features using distance $1 - |r|$.

D.5 Feature screening

Table 21 – Top ten features according to the combined screening score.

Feature	ANOVA rank	MI rank	Logistic rank	Combined rank
short	2	6	1	1
self	1	16	2	2
practice	4	2	4	3
path	3	13	3	4
human	5	4	6	5
shape	6	11	5	6
complexity	62	1	62	7
body	10	5	21	8
color	14	8	8	9
biomotion	9	9	13	10



(a) ANOVA. (b) Mutual information. (c) Combined screening.

Figure 41 – Top-ranked features under the univariate and combined screening criteria.

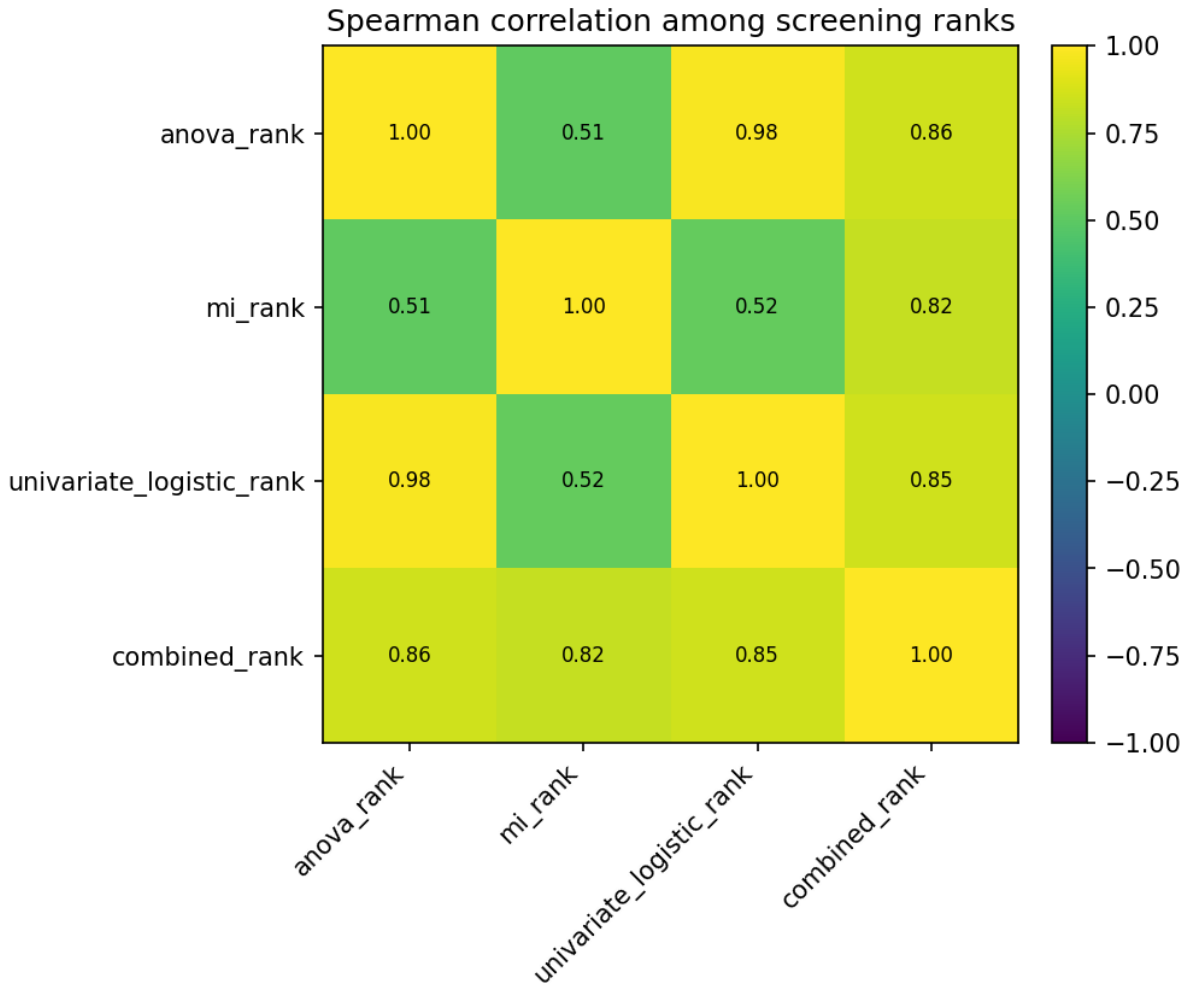


Figure 42 – Rank agreement among the feature-screening criteria.

D.6 Regularized models and sparsity analysis

Table 22 – Regularized multinomial models: train and test performance.

Model	Split	Accuracy	Balanced accuracy	Macro F1	Weighted F1
LASSO	Train	1.000	1.000	1.000	1.000
LASSO	Test	0.953	0.919	0.890	0.955
Elastic Net	Train	1.000	1.000	1.000	1.000
Elastic Net	Test	0.953	0.919	0.890	0.955

Table 23 – Sparsity summary for LASSO and Elastic Net coefficient matrices.

Model	Total coefficients	Zero coefficients	Nonzero coefficients	Zero share	Active features
LASSO	195	23	172	0.118	65
Elastic Net	195	3	192	0.015	65

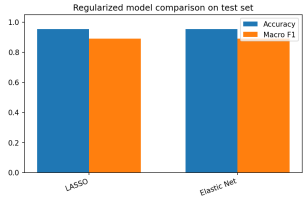
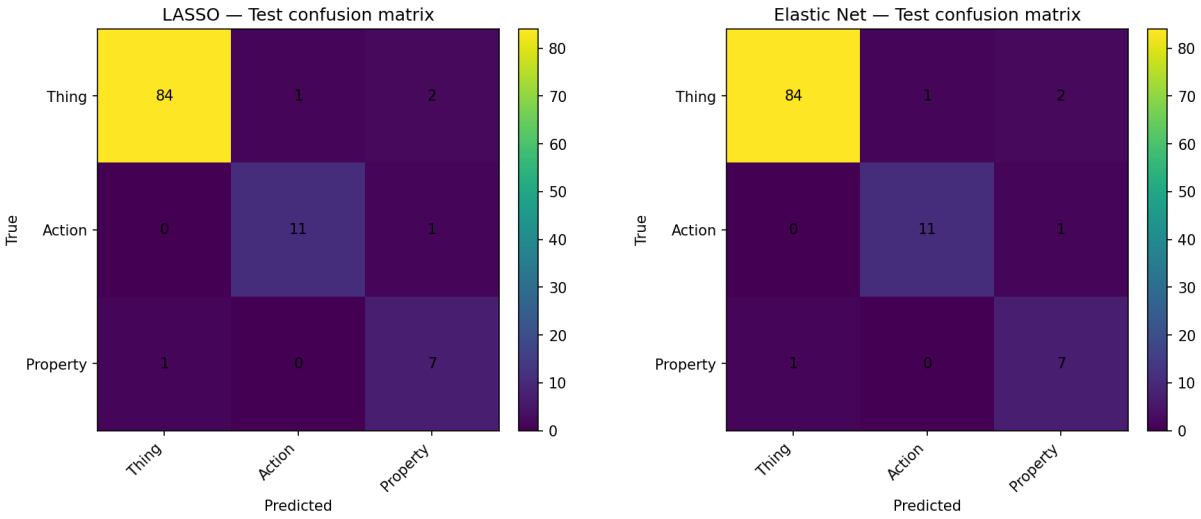


Figure 43 – Comparison of regularized multinomial models.



(a) LASSO. (b) Elastic Net.

Figure 44 – Test-set confusion matrices for the regularized models.

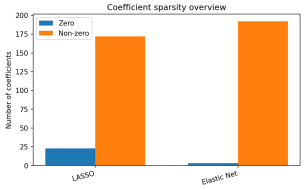
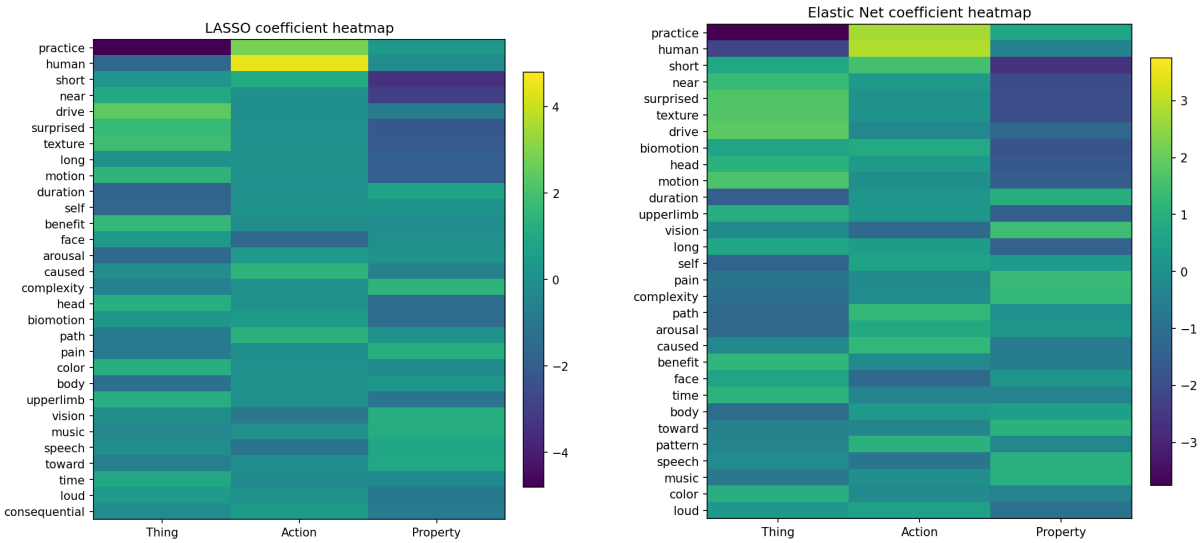


Figure 45 – Sparsity overview for the regularized multinomial coefficient matrices.



(a) LASSO coefficients. (b) Elastic Net coefficients.

Figure 46 – Coefficient heatmaps for the regularized models.

D.7 Final model evaluation and alternative classifiers

Table 24 – Test-set comparison of the main supervised models.

Model	Features	Accuracy	Balanced accuracy	Macro F1	Weighted F1
Baseline multinomial	65	0.953	0.919	0.890	0.955
LASSO	65	0.953	0.919	0.890	0.955
Elastic Net	65	0.953	0.919	0.890	0.955
Post-selection Logistic	65	0.972	0.927	0.923	0.973

Table 25 – Alternative classifier comparison on the same selected feature space, test set only.

Model	Accuracy	Balanced accuracy	Macro F1	Weighted F1
Dummy majority	0.813	0.333	0.299	0.729
Post-selection Logistic	0.944	0.915	0.875	0.947
SVM linear	0.972	0.989	0.942	0.974
SVM RBF	0.972	0.927	0.938	0.972
Random Forest	0.981	0.917	0.949	0.980
k-NN	0.953	0.819	0.883	0.950

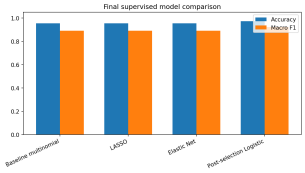
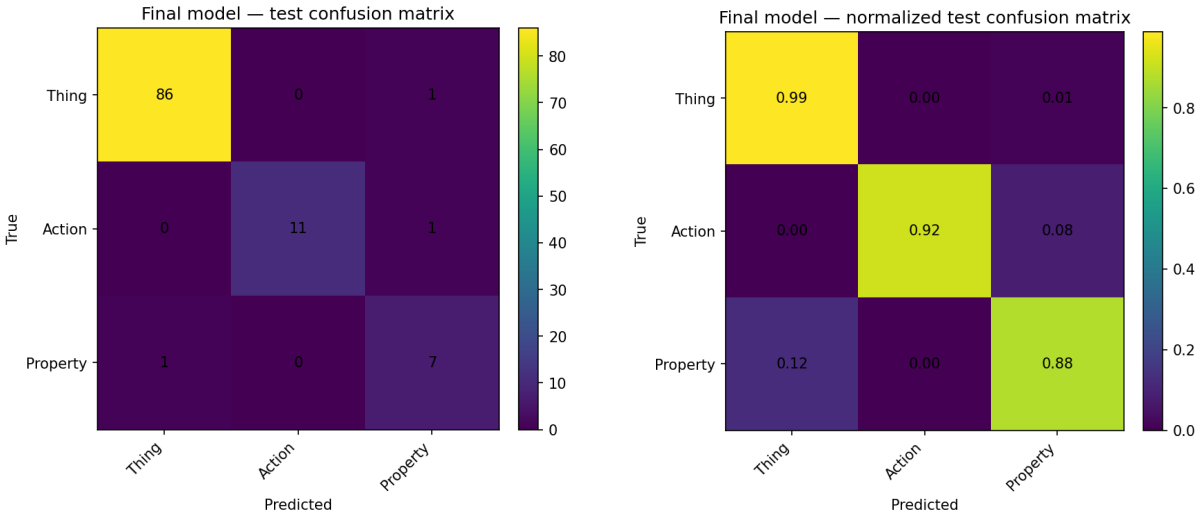


Figure 47 – Accuracy and macro-F1 comparison among the main supervised models.



(a) Absolute confusion matrix.

(b) Normalized confusion matrix.

Figure 48 – Test-set confusion matrices for the final selected model.

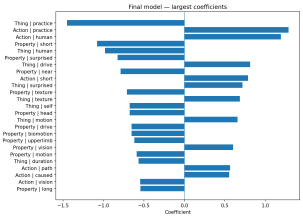
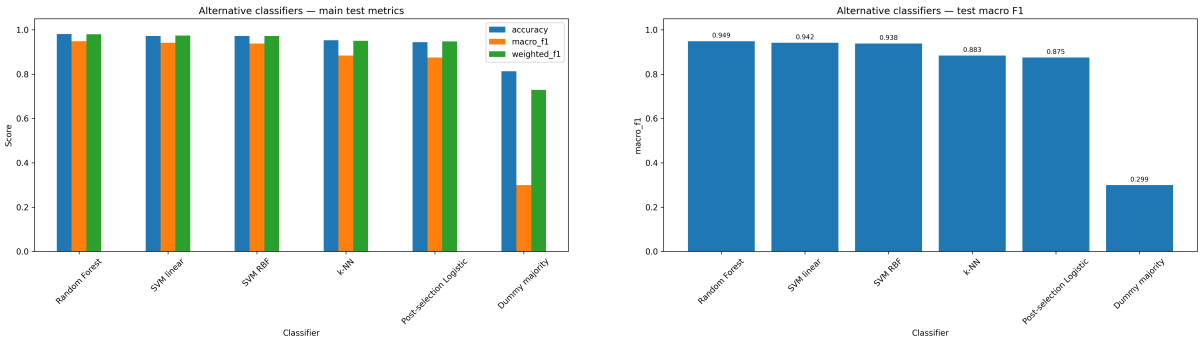


Figure 49 – Top coefficients of the final selected model.



(a) Main metrics.

(b) Macro-F1.

Figure 50 – Alternative classifier comparison on the selected semantic feature space.

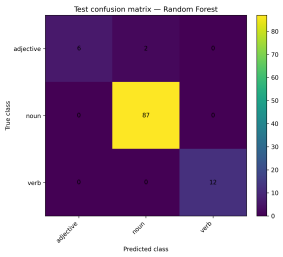


Figure 51 – Test-set confusion matrix for the Random Forest classifier, the best alternative classifier by macro-F1.

D.8 Cluster-label supervised analysis

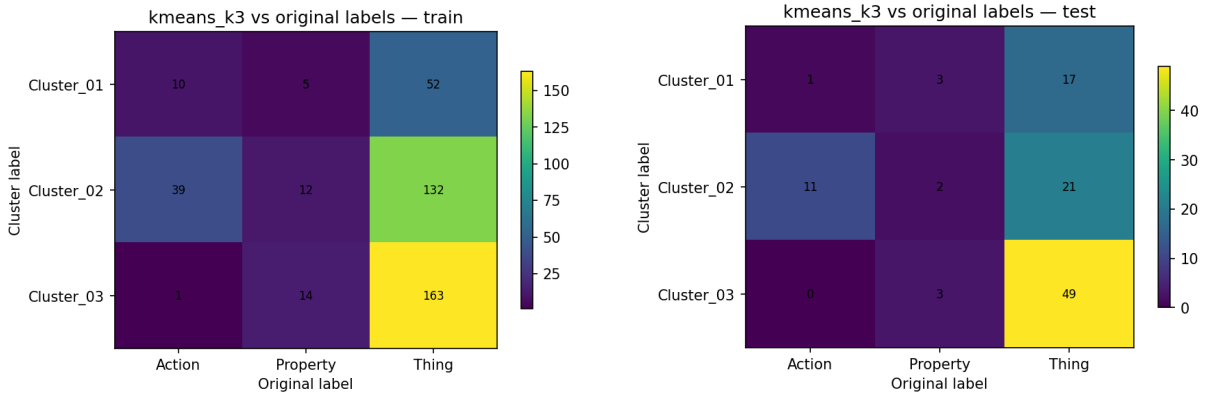
Table 26 – Distribution of the K-Means $k = 3$ cluster target used in the cluster-label supervised analysis.

Split	Cluster	Count	Proportion
Train	Cluster_01	67	0.157
Train	Cluster_02	183	0.428
Train	Cluster_03	178	0.416
Test	Cluster_01	21	0.196
Test	Cluster_02	34	0.318
Test	Cluster_03	52	0.486

Table 27 – Test-set performance of the cluster-label supervised models.

Model	Features	Accuracy	Balanced accuracy	Macro F1	Weighted F1
Baseline	65	0.981	0.974	0.979	0.981
LASSO	65	0.972	0.958	0.967	0.972
Elastic Net	65	0.981	0.974	0.979	0.981
Post-selection Logistic	53	0.981	0.974	0.979	0.981

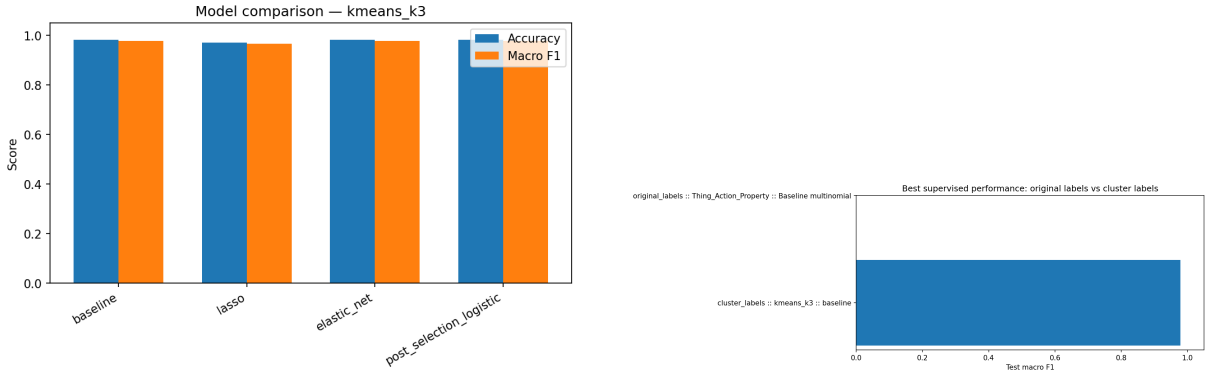
Figure 52 – Train–test distribution of the K-Means $k = 3$ cluster target.



(a) Train set.

(b) Test set.

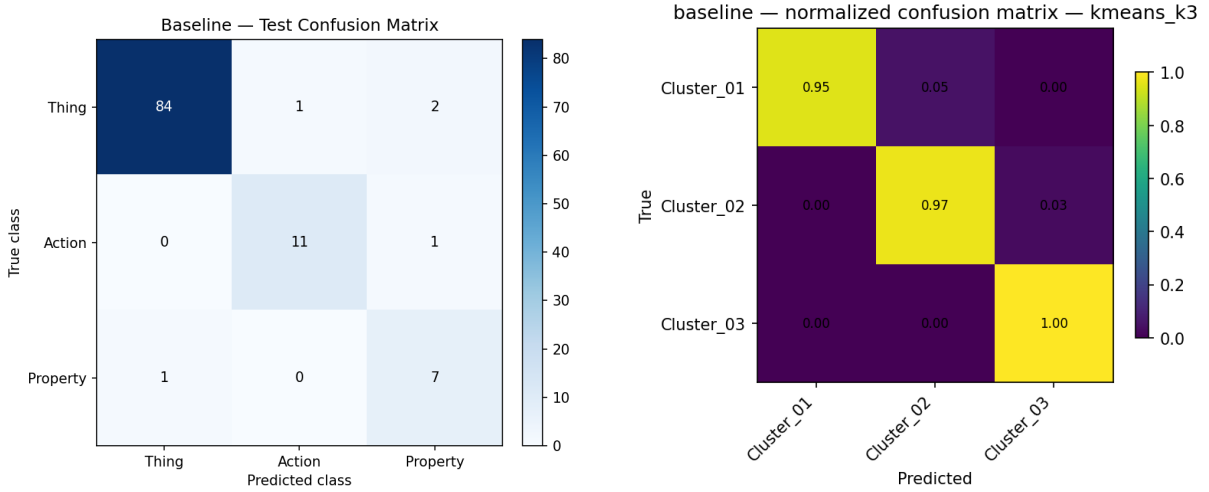
Figure 53 – Relationship between K-Means $k = 3$ cluster assignments and original lexical labels.



(a) Cluster-label models.

(b) Original labels versus cluster labels.

Figure 54 – Performance comparison for the cluster-label supervised analysis.



(a) Absolute confusion matrix.

(b) Normalized confusion matrix.

Figure 55 – Test-set confusion matrices for the best K-Means $k = 3$ cluster-label classifier.

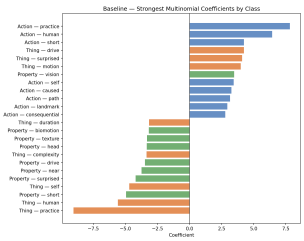
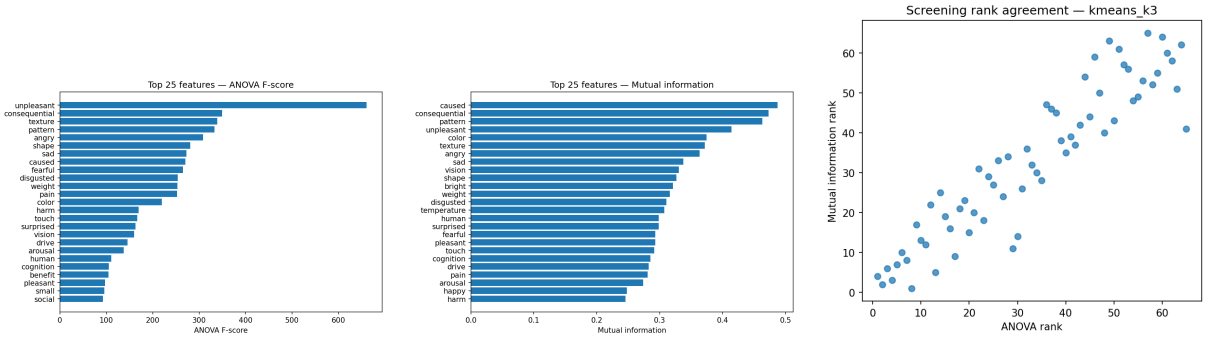


Figure 56 – Top coefficients for the best K-Means $k = 3$ cluster-label classifier.



(a) ANOVA. (b) Mutual information. (c) Screening-rank scatter.

Figure 57 – Feature-screening diagnostics for the K-Means $k = 3$ cluster target.